ACTIVIDAD EVALUATIVA

EJE 2 - Entrega 2: Desarrollo inicial del sistema de votaciones

Sistema Web de Votaciones – Festival de la Leyenda Vallenata 2027

SEBASTIÁN BAUTISTA MARTÍNEZ
MARÍA LABARCA BRICEÑO
JUAN MEJÍA MAESTRE

PROFESOR

DEIVYS MORALES URIBE

DESARROLLO WEB - 4324 - 61
INGENIERÍA DE SISTEMAS
FUNDACIÓN UNIVERSITARIA DEL ÁREA ANDINA

OCTUBRE DE 2026

AREANDINA

Tabla de Contenidos

Introducción y objetivo 2
Atención a las observaciones de la Entrega 1 4
Integrantes y distribución del trabajo 7
Desarrollo técnico 8
  Tecnologías 8
  Estructura del proyecto Django 8
  Patrón de arquitectura 9
  Conexión con MySQL 12
  Modelos y relaciones 14
  Migraciones 21
  Autenticación 23
  Administración: panel propio y Django Admin 25
Capturas de pantalla comentadas 26
  Panel de administración propio (CRUD con Django) 27
  Visualización dinámica 35
  Interfaz pública con datos de MySQL 36
  Autenticación 38
  Vistas y documentación de la API 40
  Django Admin (complementario) 41
Pruebas y resultados 44
  Pruebas automatizadas 44
  Pruebas funcionales 46
Repositorio 46
Video demostrativo 47
Conclusiones y distribución del trabajo 47
  Aprendizajes 47
  Dificultades 48
  Pendientes para la siguiente entrega 48
  Distribución del trabajo 49
Referencias 51

Introducción y objetivo

El presente documento corresponde a la Entrega 2 del proyecto de la asignatura Desarrollo Web: el desarrollo inicial del Sistema Web de Votaciones para el Festival de la Leyenda Vallenata 2027. A partir del análisis y del diseño de la Entrega 1 (actores, requerimientos, reglas de negocio, casos de uso y modelo conceptual), el equipo construyó

la aplicación con Python, Django (patrón MVT) y MySQL: vistas y plantillas de Django que muestran los datos de la base de datos, registro, inicio y cierre de sesión con el sistema de autenticación de Django, y un panel de administración propio para ediciones, categorías, votaciones y opciones, además de Django Admin.

El modelo de datos respeta la jerarquía definida en la primera entrega, Edición → Categoría → Votación → Opción, y queda preparado para incorporar el voto y sus restricciones en la siguiente entrega. Como la plataforma es configurable, ninguna categoría ni votación está escrita en el código: todas se crean desde el panel y las vistas de Django las leen de MySQL.

Objetivo de la entrega. Implementar la estructura inicial de la aplicación web con Python, Django y MySQL, integrando las primeras funcionalidades del sistema de votaciones a partir del análisis y diseño de la primera entrega.

Tabla 1

Funcionalidades mínimas de la Entrega 2 y dónde se evidencian

Funcionalidad exigida	Estado	Evidencia
Proyecto Django: aplicaciones, rutas, vistas y plantillas	Cumplido	Figuras 1, 2, 3, 4 y 15; Figuras 18 a 43
MySQL: conexión, modelos relacionados y migraciones	Cumplido	Figuras 5, 6, 7, 8, 9, 10, 11, 12, 13 y 14; Tabla 7
Autenticación: registro, inicio y cierre de sesión	Cumplido	Figuras 15, 16 y 39 a 43
Ediciones: registrar y consultar ediciones anuales	Cumplido	Figuras 19 y 20
Categorías vinculadas a una edición	Cumplido	Figuras 21 a 25, 32 a 34 y 36
Votaciones y opciones	Cumplido	Figuras 26 a 31, 38 y 47
Interfaz pública con datos reales de MySQL (vistas y plantillas Django)	Cumplido	Figuras 34 a 38

AREANDINA

Atención a las observaciones de la Entrega 1

Al revisar la Entrega 1, el docente pidió cuatro correcciones. Siguiendo el primer paso de la ruta de desarrollo de esta actividad, el equipo las revisó antes de programar. La Tabla 2 resume cómo se atendió cada una; la Tabla 3 conserva la relación entre requerimientos, casos de uso, interfaces y modelo de datos, y la Tabla 4 separa las reglas de negocio según quién las propuso y si ya están aprobadas.

Tabla 2

Observaciones de la Entrega 1 y cómo se atendieron

Observación del docente	Cómo se atendió en la Entrega 2	Evidencia
Diferenciar las reglas de negocio propuestas por el equipo de las que requieren aprobación de la Fundación.	Cada regla conserva su origen y su estado de la Entrega 1. Las que esperan aprobación se programaron como opciones configurables desde el panel (votos por usuario, visibilidad de resultados, pedir el documento, entre otras), con un valor por defecto que la Fundación puede cambiar sin tocar el código.	Tabla 4
Conservar la relación entre requerimientos, casos de uso, interfaces y modelo de datos.	Cada requerimiento de la Entrega 1 se enlaza con su caso de uso, la pantalla o ruta de Django que lo implementa, el modelo que usa y la figura que lo demuestra. Los mismos identificadores (RF, CU y RN) se usan en el código y en la documentación de la API.	Tabla 3
Garantizar que la administración sea configurable y no dependa de información fija en el código.	Ediciones, categorías, votaciones y opciones se crean, editan y eliminan desde el panel propio y se guardan en MySQL. El sitio consulta la edición activa con el ORM, por lo que una categoría nueva aparece sin cambiar el código.	Figuras 18 a 34
Respaldar el desarrollo con evidencias de funcionamiento en Django y MySQL.	Se documentan la conexión, los modelos, las migraciones, la autenticación y las capturas con datos de MySQL, además de 219 pruebas automatizadas que se ejecutan contra MySQL.	Figuras 5 a 16 y 49; Tablas 8 y 9

Nota. Elaboración propia a partir de la retroalimentación del docente a la Entrega 1.

AREANDINA

Tabla 3

Trazabilidad de los requerimientos de la Entrega 1 en la Entrega 2

Requerimiento	Caso de uso	Pantalla o ruta (Django)	Modelo	Estado	Evidencia
RF-01 Registro de votantes	CU-01	/registro	Usuario	Implementado	Figuras 39 y 40
RF-02 Inicio y cierre de sesión por rol	CU-02	/login · /logout	Usuario (rol)	Implementado	Figuras 41 a 43
RF-03 Recuperar la contraseña	CU-11	/recuperar-clave	Usuario	Implementado (enlace por correo)	Código: portal/vistas/cuentas.py
RF-04 Categorías de la edición vigente	CU-03	/{año}	Edicion, Categoria	Implementado	Figuras 35 y 36
RF-05 Votaciones por categoría con estado y filtro	CU-03	/{año}/categorias/{categoría}	Votacion	Implementado	Figura 37
RF-06 Detalle de la votación y sus opciones	CU-03	/{año}/categorias/{categoría}/{votación}	Votacion, Opcion	Implementado	Figura 38
RF-07 Seleccionar y confirmar el voto	CU-04	…/{votación}/votar	Voto	Avance; se completa en la Entrega 3	Figuras 7 y 8
RF-08 Validaciones antes de votar	CU-04	servicio emitir_voto	Voto, Votacion	Avance; reglas programadas	Tabla 4
RF-09 Comprobante e historial de votos	CU-04, CU-10	…/comprobante · /mis-votos	Voto	Avance; se completa en la Entrega 3	Figura 11 (código único)
RF-10 Gestionar ediciones	CU-09	/panel/ediciones	Edicion	Implementado	Figuras 19 y 20
RF-11 Gestionar categorías	CU-05	/panel/categorias	Categoria	Implementado	Figuras 21 a 25, 32 y 33
RF-12 Gestionar votaciones	CU-06	/panel/votaciones	Votacion	Implementado	Figuras 26 a 28
RF-13 Gestionar opciones	CU-07	/panel/votaciones/{id}/opciones	Opcion	Implementado	Figuras 29 a 31
RF-14 Resultados con tabla y gráfico	CU-08	API: resultados de la votación	Voto	Parcial; tabla, gráfico y exportación en la Entrega 3	Figura 44
RF-15 Publicar resultados*	CU-08, CU-12	Detalle de la votación · API	Votacion, Edicion	Configurable, pendiente de validación	Figura 44
RF-16 Auditoría de acciones administrativas	CU-08, CU-13	/panel (actividad reciente)	RegistroAuditoria	Implementado	Figura 18
RF-17 Gestionar administradores*	—	/django-admin	Usuario (rol)	Parcial, desde Django Admin	Figura 48
RF-18 Verificar el correo del votante*	CU-01	/confirmar-correo	Usuario	Implementado como propuesta (RN-18)	Figura 40

Nota. Elaboración propia. Los identificadores RF, CU y RN son los de la Entrega 1; el asterisco (*) marca los requerimientos «Pendiente de validación». «Avance» indica funciones que ya tienen modelo y lógica, pero cuyo cierre corresponde a la Entrega 3 según el alcance de la actividad.

Tabla 4

Reglas de negocio según su origen y su aplicación en la Entrega 2

Regla	Descripción (Entrega 1)	Origen	Aplicación en la Entrega 2
RN-01	Toda votación pertenece a una categoría y toda categoría a una edición.	Confirmada (enunciado)	Llaves foráneas en MySQL (Figura 11).
RN-10	Para registrarse se debe aceptar la política de datos (Ley 1581 de 2012).	Confirmada (normativa)	Casilla obligatoria del registro (Figura 39).
RN-02	Solo los usuarios autenticados pueden votar.	Propuesta del equipo	La vista de voto exige sesión iniciada.
RN-03	Solo se vota entre la apertura y el cierre.	Propuesta del equipo	emitir_voto rechaza votaciones que no están abiertas.
RN-06	Una votación se publica solo con dos opciones activas.	Propuesta del equipo	El panel lo valida al publicar (Figura 28).
RN-09	No se eliminan votaciones ni opciones con votos.	Propuesta del equipo	Se impide el borrado; se cierran o desactivan.
RN-12	Las acciones de los administradores quedan en la auditoría.	Propuesta del equipo	Tabla registro_auditoria (Figura 18).
RN-15	No se agregan opciones a una votación cerrada ni categorías a una edición cerrada.	Propuesta del equipo	Validado en el panel y en la API.
RN-16	El administrador puede pausar las votaciones de una edición.	Propuesta del equipo	Campo votaciones_pausadas; el voto se rechaza en pausa.
RN-18	Solo votan usuarios con el correo confirmado.	Propuesta del equipo	Campo correo_verificado y enlace de confirmación.
RN-19	Desde que una votación abre o tiene votos, sus opciones no se modifican.	Propuesta del equipo (nueva)	Bloqueo de opciones en el panel y en la API.
RN-04	Límite de votos por usuario por edición, personalizable por votación.	Requiere aprobación de la Fundación	Configurable; valor por defecto: un voto.
RN-05	Un voto confirmado no se modifica ni se anula.	Requiere aprobación de la Fundación	Provisional: no existe opción para editar votos.
RN-07	Visibilidad de los resultados para el público.	Requiere aprobación de la Fundación	Configurable: en tiempo real, al cierre o no publicar.
RN-08	La votación del público no reemplaza la decisión del jurado.	Requiere aprobación de la Fundación	Sin efecto en el código; se espera el texto oficial.
RN-11	Posibles restricciones de edad o de territorio.	Requiere aprobación de la Fundación	No implementada hasta conocer la decisión.
RN-13	Las votaciones cerradas siguen visibles unos días.	Requiere aprobación de la Fundación	Configurable (días visibles después del cierre).
RN-14	Las ediciones cerradas no se muestran al público.	Requiere aprobación de la Fundación	Aplicada en las consultas del sitio; se puede ajustar.
RN-17	Documento de identidad solo si la administración lo activa.	Requiere aprobación de la Fundación	Configurable; desactivado por defecto.

Nota. Elaboración propia. «Requiere aprobación de la Fundación» corresponde a las reglas marcadas como «Pendiente de validación» en la Entrega 1: se programaron como opciones configurables para no fijar en el código una decisión que todavía es del cliente. RN-19 es una propuesta nueva del equipo en esta entrega.

Integrantes y distribución del trabajo

El equipo está conformado por tres integrantes. Cada uno registra sus aportes con su propia cuenta en el repositorio de GitHub, lo que deja trazabilidad de quién hizo cada parte. La distribución detallada de responsabilidades se presenta en la sección de conclusiones (Tabla 10).

Tabla 5

Integrantes del equipo

Integrante	Usuario de GitHub	Rol principal
Juan Mejía Maestre	juandiegows	Coordinación, frontend y despliegue
Sebastián Bautista Martínez	sbautista15	Backend Django, modelos y base de datos
María Labarca Briceño	mlabarca-jpg	Documentación, pruebas y administración

Nota. La actividad indica grupos de cuatro estudiantes; este equipo está conformado por tres integrantes, por lo que cada uno asume más de un frente de trabajo.

Desarrollo técnico

Tecnologías

Tabla 6

Tecnologías utilizadas

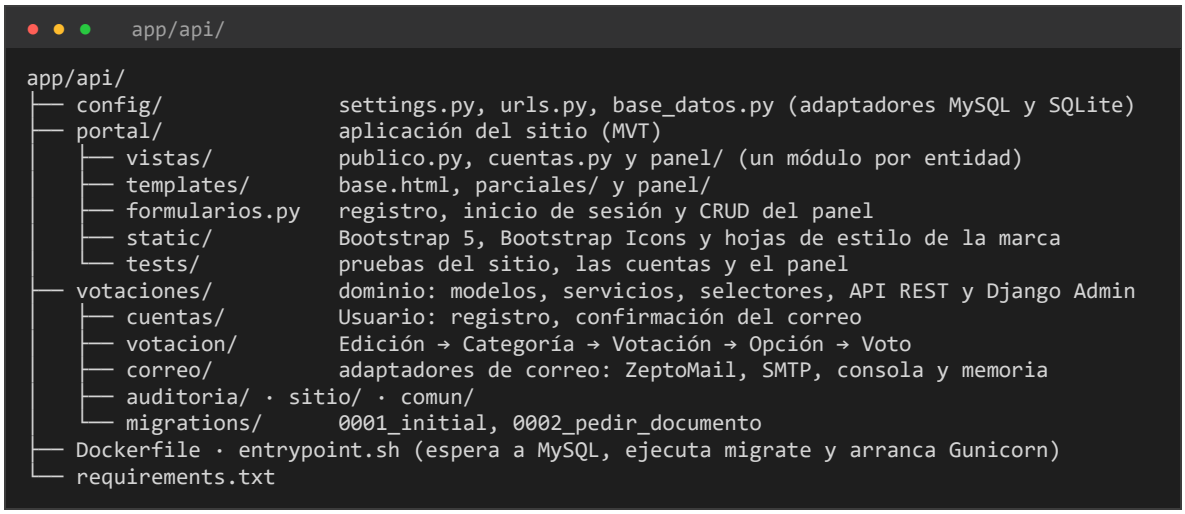
Capa	Tecnología	Uso en el proyecto
Backend	Python 3.12, Django 5.2	Modelos, migraciones, autenticación, vistas y plantillas (MVT), Django Admin y reglas de negocio
Interfaz (MVT)	Plantillas de Django, HTML5, CSS3, Bootstrap 5.3, Bootstrap Icons y JavaScript	Sitio público, registro e inicio de sesión y panel de administración propio
Base de datos	MySQL 9.7 en producción y 8.4 en Docker local (conector PyMySQL)	Persistencia de todas las entidades mediante el ORM de Django
API REST	Django REST Framework 3.18	Las mismas reglas expuestas en JSON para otros clientes (la web React del repositorio)
Correo	ZeptoMail (Zoho) o SMTP	Confirmación del correo y bienvenida, con adaptadores intercambiables
Entorno	Docker Compose, Git y GitHub	MySQL y Django en contenedores; control de versiones e integración continua

Estructura del proyecto Django

El backend vive en app/api. El proyecto se llama config (configuración, rutas raíz, adaptador de base de datos y servidor WSGI) y tiene dos aplicaciones: portal, que contiene las vistas, formularios y plantillas del sitio (patrón MVT), y votaciones, que agrupa por dominio los modelos, las reglas de negocio, la API REST y Django Admin. Las dos comparten los mismos modelos y servicios, por lo que ninguna regla se escribe dos veces.

Figura 1

Estructura de carpetas del backend



Nota. Elaboración propia a partir del repositorio. Se resumen las carpetas principales.

Patrón de arquitectura

Django organiza la aplicación en Modelo, Vista y Plantilla (MVT), y así quedó construida esta entrega:

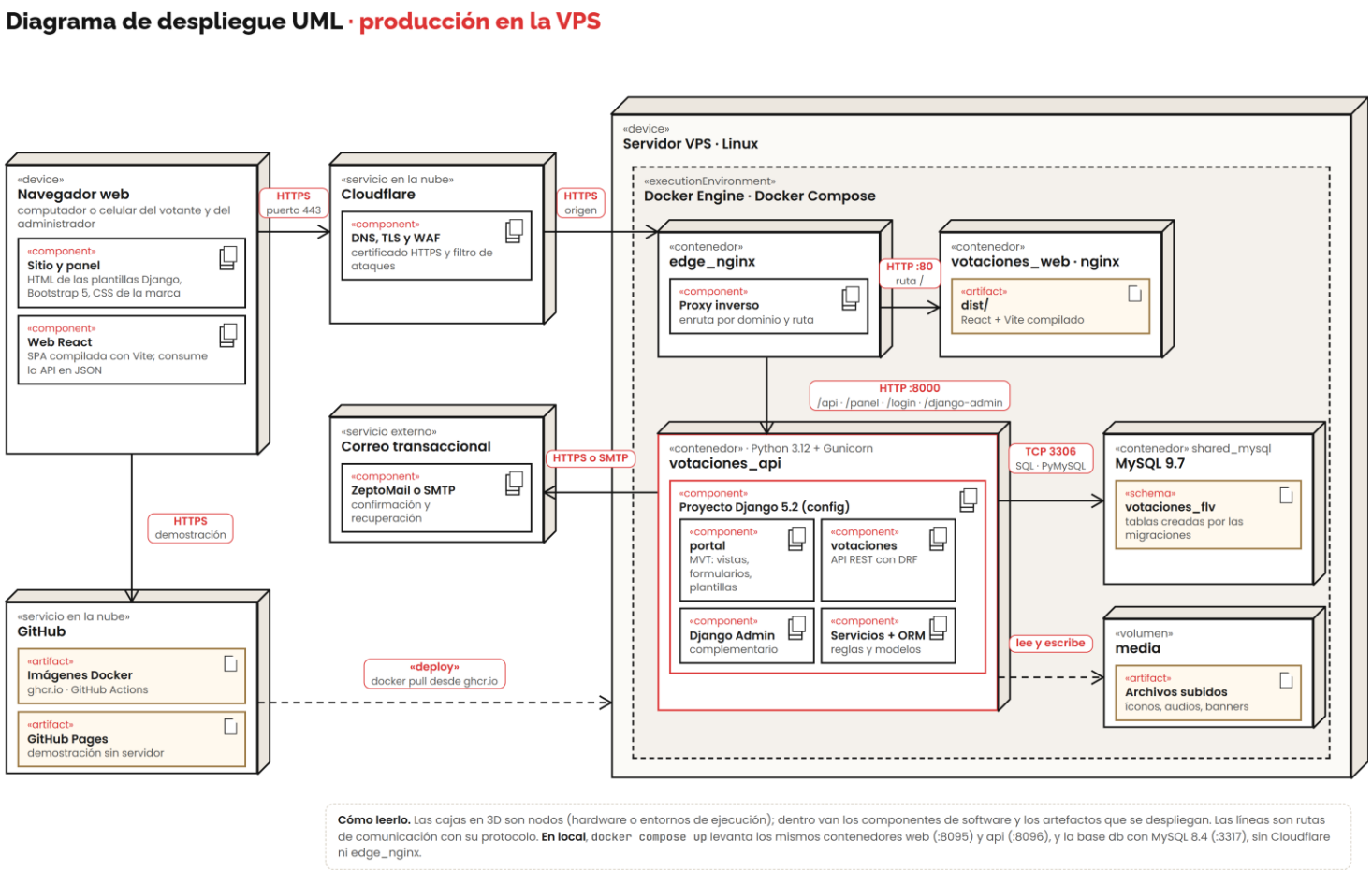
- Modelo: las clases de votaciones/votacion/models.py definen las tablas de MySQL y sus relaciones; las consultas se hacen con el ORM de Django, sin escribir SQL.
- Vista y plantilla: las funciones de portal/vistas/ reciben cada petición, consultan los modelos y entregan los datos a una plantilla de templates/portal/, que genera el HTML con el diseño de la marca (Bootstrap y las hojas de estilo propias). El panel /panel usa el mismo patrón con formularios de Django.

Las reglas de negocio están en una capa de servicios (servicios.py) y las consultas en selectores, de modo que las vistas del sitio, la API REST y Django Admin usan las mismas reglas. La base de datos y el correo se conectan con adaptadores intercambiables (patrón puerto/adaptador). El flujo de una petición es: navegador → urls.py → vista de Django →

servicio o selector → modelo (ORM) → MySQL → plantilla → HTML. La Figura 2 muestra cómo se despliega el sistema (diagrama UML de despliegue): navegador, Cloudflare, contenedores Docker de la web, la API Django y MySQL.

Figura 2

Diagrama de despliegue UML del sistema en producción



Nota. Elaboración propia a partir del repositorio y de la configuración de Docker. Los nodos en 3D son el hardware y los entornos de ejecución; dentro van los componentes y artefactos. En local, docker compose levanta los mismos contenedores web, api y db sin Cloudflare ni edge_nginx.

Figura 3

Rutas raíz del proyecto

app/api/config/urls.py

```
urlpatterns = [
    path("django-admin/", admin.site.urls),        # Django Admin
    path("api/", include("votaciones.urls")),       # API REST
    path("", include("portal.urls")),               # sitio Django (MVT)
]
```

Nota. Fragmento del repositorio (app/api/config/urls.py). Se omiten líneas que no son necesarias para la explicación.

Figura 4

Rutas del sitio: autenticación, panel y páginas públicas

app/api/portal/urls.py

```
path("registro", cuentas.registro, name="registro"),
path("login", cuentas.iniciar_sesion, name="login"),
path("logout", cuentas.cerrar_sesion, name="logout"),
path("panel/categorias", categorias.categorias, name="panel-categorias"),
path("<anio:anio>", publico.edicion, name="edicion"),
path("<anio:anio>/categorias/<slug:categoria>", publico.categoria, name="categoria"),
```

Nota. Fragmento del repositorio (app/api/portal/urls.py). Se omiten líneas que no son necesarias para la explicación.

Conexión con MySQL

La conexión se configura con un adaptador (config/base_datos.py): la variable DB_ENGINE=mysql activa AdaptadorMySQL, que usa el conector PyMySQL, la codificación utf8mb4 (tildes y ñ), el modo estricto y conexiones persistentes. Los datos de acceso se leen de variables de entorno, de modo que ninguna contraseña queda escrita en el código. Para desarrollo rápido sin Docker, AdaptadorSQLite usa un archivo local; el resto del sistema solo conoce el ORM. En producción, la API se conecta a MySQL 9.7 en la VPS;

el entorno Docker local usa MySQL 8.4, que es donde se tomaron las evidencias de este informe. El esquema es el mismo en ambos porque lo crean las mismas migraciones.

Figura 5

Adaptador de base de datos para MySQL

app/api/config/base_datos.py

```python
class AdaptadorMySQL(AdaptadorBaseDatos):
    def preparar(self):
        import pymysql
        pymysql.install_as_MySQLdb()

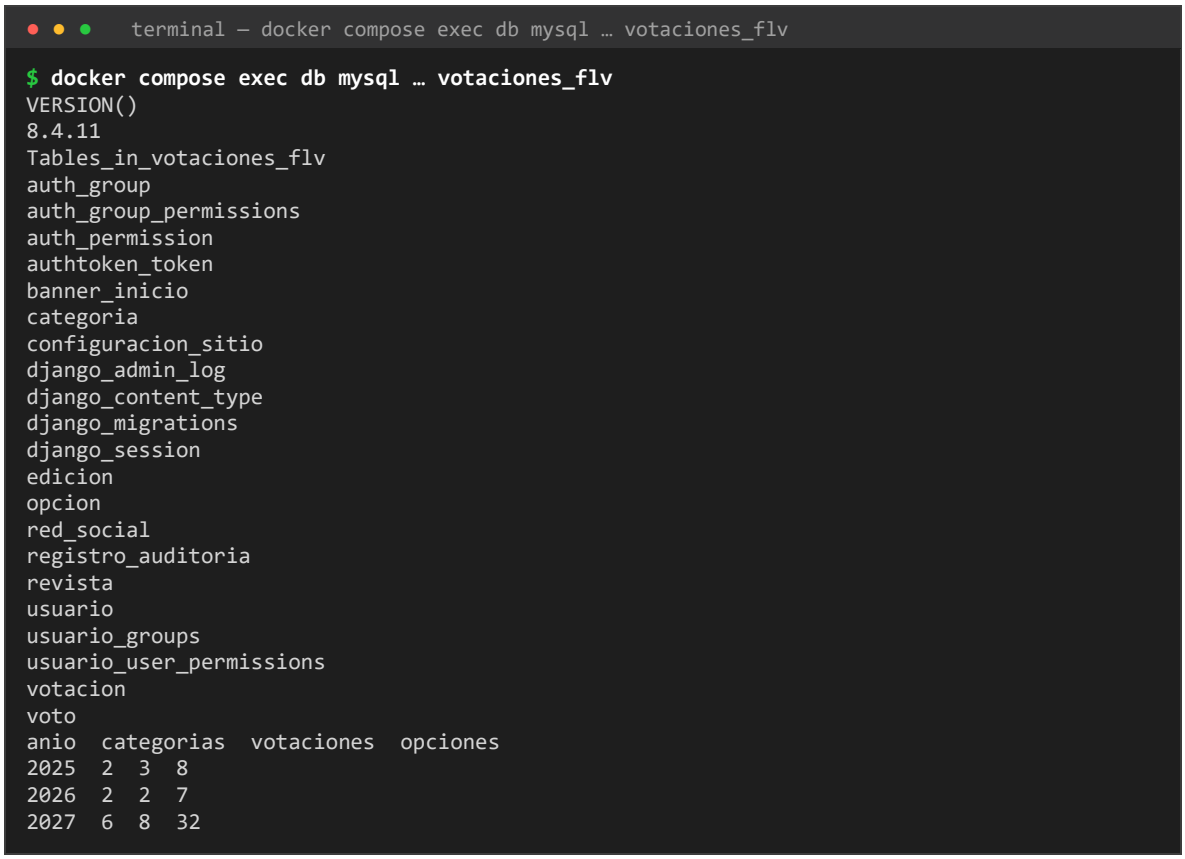
    def configuracion(self, base_dir):
        return {
            "ENGINE": "django.db.backends.mysql",
            "NAME": os.getenv("DB_NAME", "votaciones_flv"),
            "USER": os.getenv("DB_USER", "votaciones"),
            "PASSWORD": os.getenv("DB_PASSWORD", ""),
            "HOST": os.getenv("DB_HOST", "127.0.0.1"),
            "CONN_MAX_AGE": 60, "CONN_HEALTH_CHECKS": True,
            "OPTIONS": {"charset": "utf8mb4",
                        "init_command": "SET sql_mode='STRICT_TRANS_TABLES'"},
        }

# config/settings.py
DATABASES = {"default": base_datos.configurar(os.getenv("DB_ENGINE", "sqlite"), BASE_DIR)}
```

Nota. Fragmento del repositorio (app/api/config/base_datos.py). Los valores reales de usuario y contraseña se entregan al contenedor como variables de entorno y no se incluyen en este documento.

Figura 6

Versión de MySQL, tablas creadas por las migraciones y datos de prueba por edición



Nota. Salida de consola tomada el 18 de octubre de 2026 en el entorno Docker local. La edición 2027 incluye la categoría temporal creada durante las pruebas.

Modelos y relaciones

Los modelos implementan la jerarquía del modelo conceptual de la Entrega 1. Cada relación es una llave foránea con `on_delete=PROTECT`, que impide borrar una edición con categorías o una votación con opciones y votos. Las reglas de integridad se declaran como restricciones de la base de datos (por ejemplo, la fecha de cierre debe ser posterior a la de apertura y el nombre de la categoría no se repite dentro de la misma edición). El diagrama entidad-relación de la Figura 7 muestra las tablas que generan las migraciones; el diagrama

de clases UML de la Figura 8 muestra los modelos, su herencia y sus métodos, y las Figuras 11 y 12 comprueban en MySQL las llaves foráneas, las restricciones y los datos relacionados.

Tabla 7

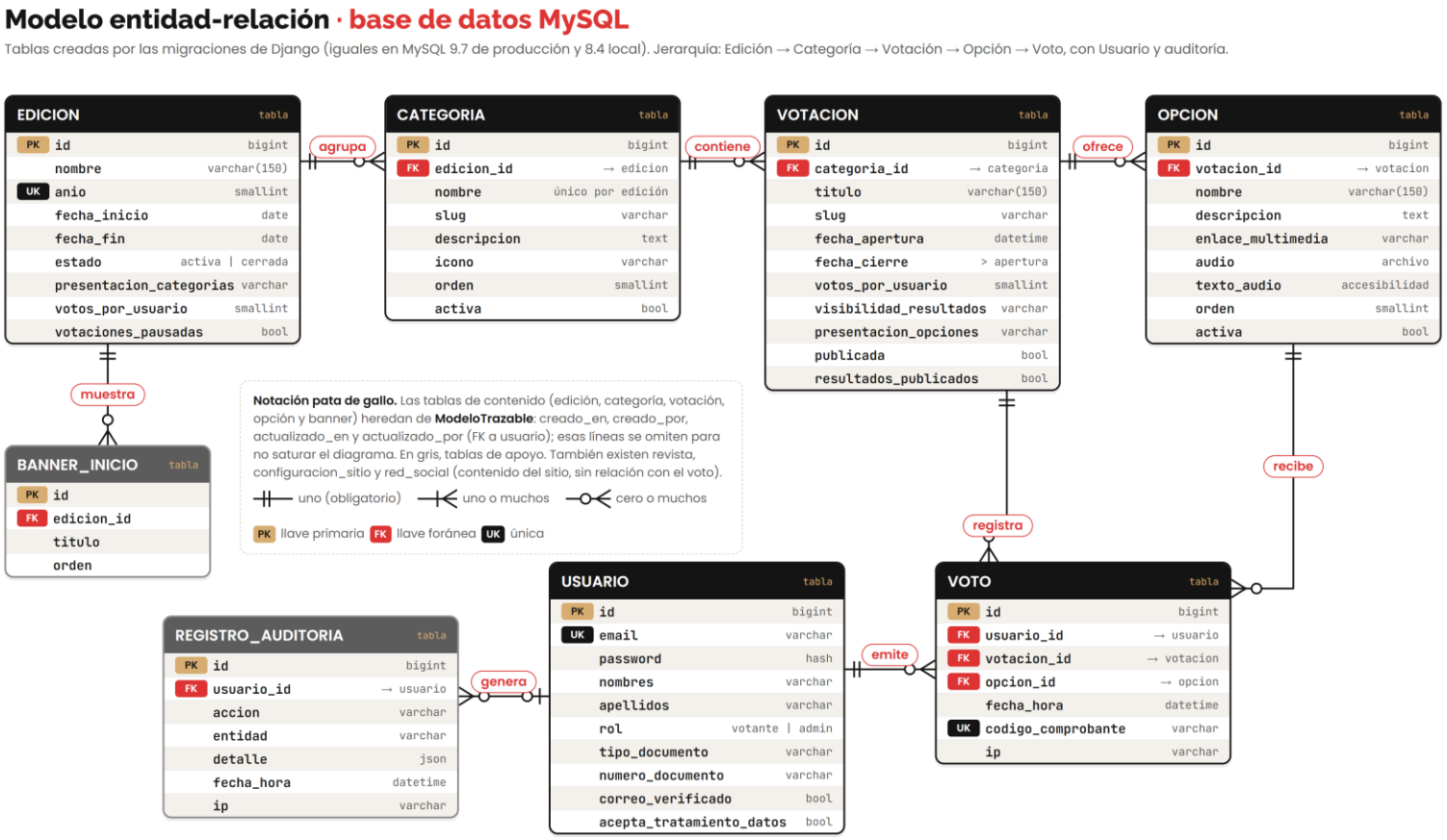
Modelos principales, relaciones y cardinalidades

Modelo (tabla)	Campos principales	Relación
Edicion (`edicion`)	nombre, anio (único), fecha_inicio, fecha_fin, estado, votos_por_usuario, visibilidad_resultados	1 edición → N categorías
Categoria (`categoria`)	edicion, nombre, slug, descripcion, icono, activa, orden	N categorías → 1 edición; 1 categoría → N votaciones
Votacion (`votacion`)	categoria, titulo, slug, fecha_apertura, fecha_cierre, votos_por_usuario, publicada	N votaciones → 1 categoría; 1 votación → N opciones
Opcion (`opcion`)	votacion, nombre, descripcion, imagen, enlace_multimedia, audio, orden, activa	N opciones → 1 votación
Usuario (`usuario`)	email (único, se usa para iniciar sesión), nombres, apellidos, rol, correo_verificado	1 usuario → N votos
Voto (`voto`)	usuario, votacion, opcion, fecha_hora, codigo_comprobante (único)	Preparado para la Entrega 3: une usuario, votación y opción

Nota. Todos los modelos heredan de ModeloTrazable, que agrega creado_en, creado_por, actualizado_en y actualizado_por.

Figura 7

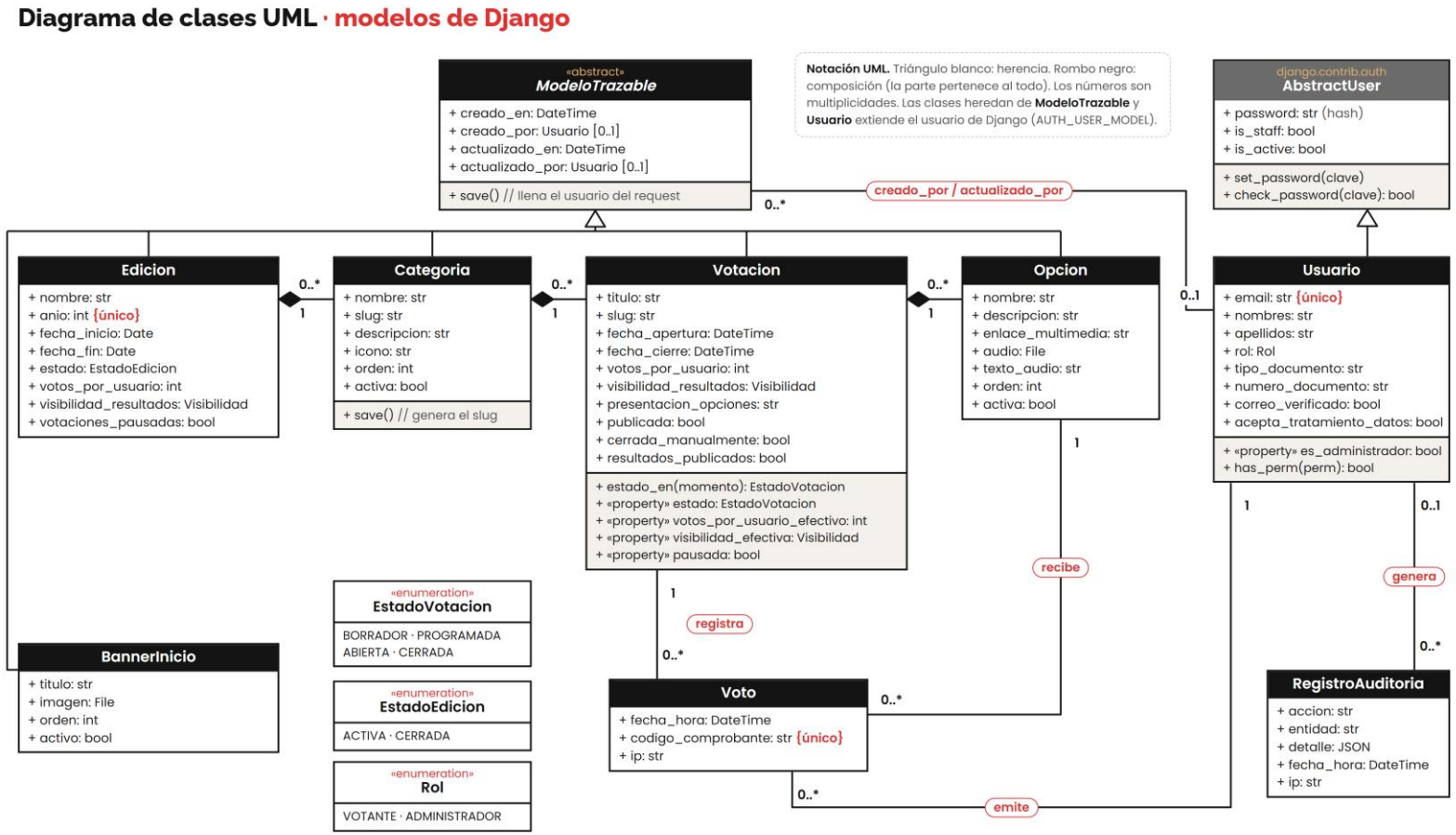
Modelo entidad-relación de la base de datos MySQL



Nota. Elaboración propia a partir de los modelos de Django y de las tablas de MySQL. Las líneas de creado_por y actualizado_por hacia usuario se omiten para no saturar el diagrama.

Figura 8

Diagrama de clases UML de los modelos de Django



Nota. Elaboración propia a partir de app/api/votaciones. Se muestran los atributos y métodos principales; las clases de contenido heredan de ModeloTrazable y Usuario extiende el usuario de Django.

Figura 9

Modelos Edición y Categoría (relación uno a muchos)

app/api/votaciones/votacion/models.py

```python
class Edicion(ModeloTrazable):
    nombre = models.CharField(max_length=150)
    anio = models.PositiveSmallIntegerField("año", unique=True)
    fecha_inicio = models.DateField()
    fecha_fin = models.DateField()
    estado = models.CharField(max_length=10, choices=Estado.choices,
default=Estado.ACTIVA)
    votos_por_usuario = models.PositiveSmallIntegerField(default=1)

    class Meta:
        db_table = "edicion"
        constraints = [models.CheckConstraint(
            condition=Q(fecha_fin__gte=F("fecha_inicio")), name="edicion_fechas_validas")]



class Categoria(ModeloTrazable):
    edicion = models.ForeignKey(Edicion, on_delete=models.PROTECT,
related_name="categorias")
    nombre = models.CharField(max_length=120)
    slug = models.SlugField(max_length=120, blank=True)
    activa = models.BooleanField(default=True)
    orden = models.PositiveSmallIntegerField(default=0)

    class Meta:
        db_table = "categoria"
        constraints = [models.UniqueConstraint(
            fields=["edicion", "nombre"], name="categoria_nombre_unico_por_edicion")]
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/models.py). Se omiten líneas que no son necesarias para la explicación.

Figura 10

Modelos Votación y Opción

app/api/votaciones/votacion/models.py

```python
class Votacion(ModeloTrazable):
    categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT,
related_name="votaciones")
    titulo = models.CharField(max_length=150)
    fecha_apertura = models.DateTimeField()
    fecha_cierre = models.DateTimeField()
    votos_por_usuario = models.PositiveSmallIntegerField(default=1)
    publicada = models.BooleanField(default=False)

    class Meta:
        db_table = "votacion"
        constraints = [models.CheckConstraint(
            condition=Q(fecha_cierre__gt=F("fecha_apertura")),
            name="votacion_cierre_posterior_apertura")]

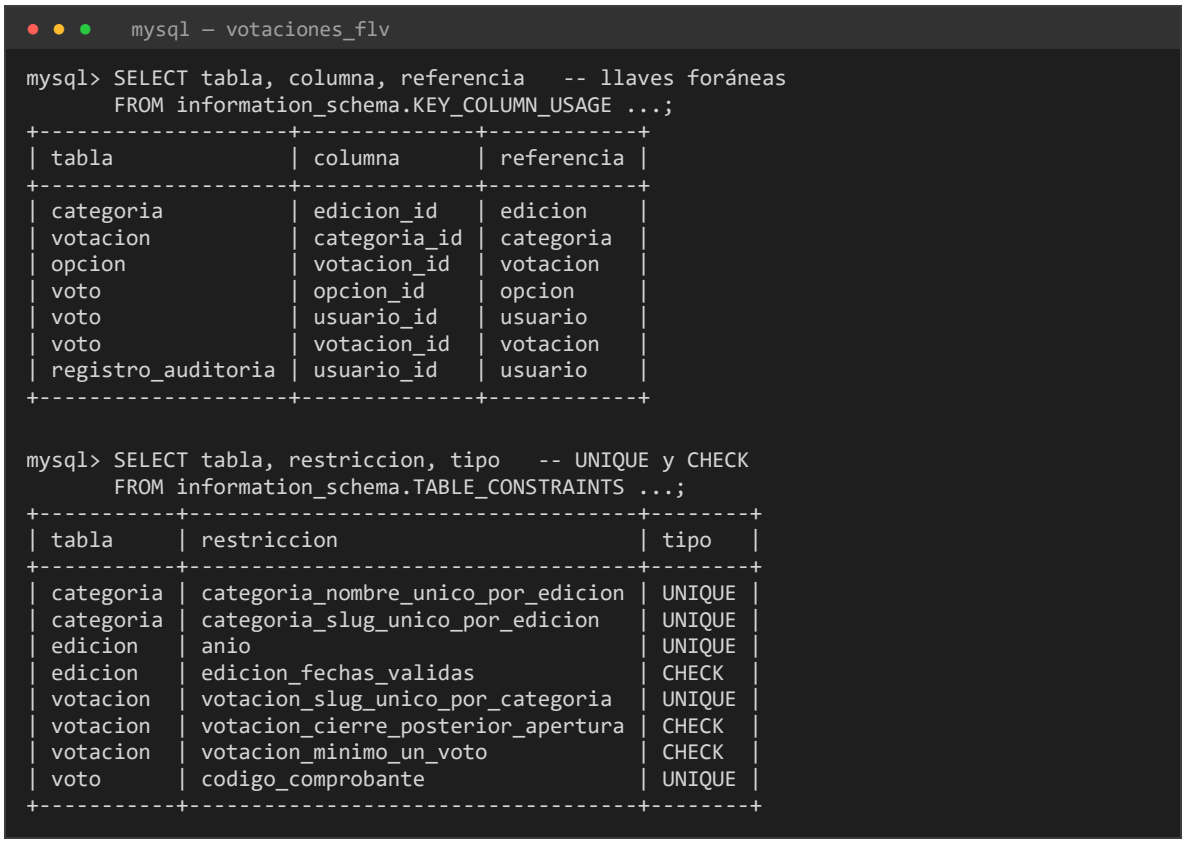


class Opcion(ModeloTrazable):
    votacion = models.ForeignKey(Votacion, on_delete=models.PROTECT,
related_name="opciones")
    nombre = models.CharField(max_length=150)
    descripcion = models.TextField(blank=True)
    orden = models.PositiveSmallIntegerField(default=0)
    activa = models.BooleanField(default=True)
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/models.py). Se omiten líneas que no son necesarias para la explicación.

Figura 11

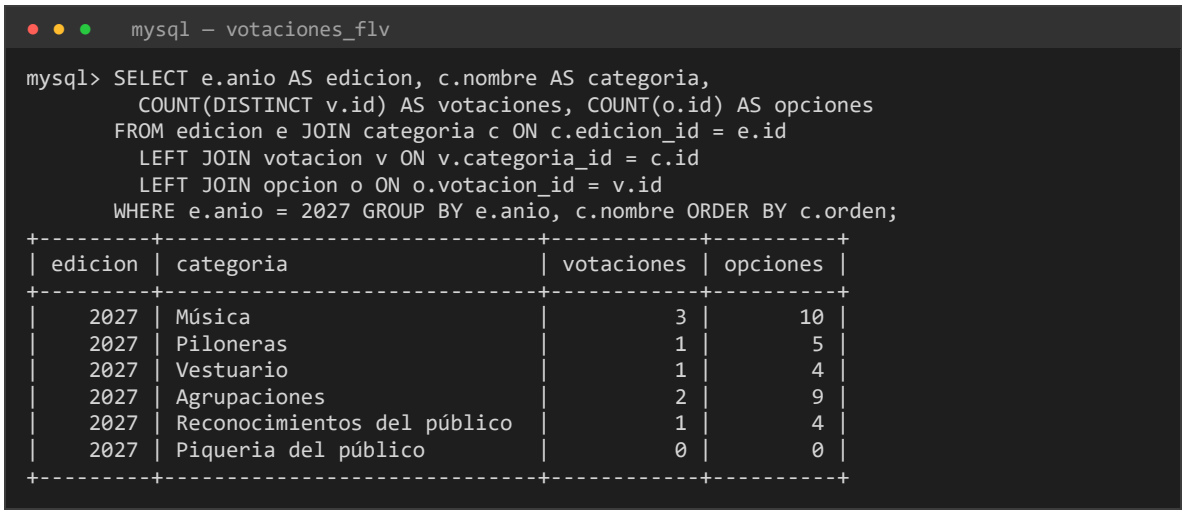
Llaves foráneas y restricciones creadas en MySQL por las migraciones



Nota. Consulta a information_schema tomada el 24 de octubre de 2026 en el entorno Docker local con MySQL 8.4. Cada relación del modelo es una llave foránea real en MySQL, y las reglas de los modelos (nombres únicos por edición, cierre posterior a la apertura, al menos un voto por usuario) quedan como restricciones UNIQUE y CHECK.

Figura 12

Datos relacionados de la edición 2027: Edición → Categoría → Votación → Opción



Nota. Consulta con JOIN tomada el 24 de octubre de 2026 en el entorno Docker local con MySQL 8.4. Demuestra que la jerarquía funciona con datos persistidos en MySQL; los datos son de prueba.

Migraciones

Las tablas se crean con el sistema de migraciones de Django. Al arrancar, el contenedor de la API espera a que MySQL esté disponible y ejecuta `python manage.py migrate` (archivo `entrypoint.sh`). En la integración continua de GitHub se ejecuta `makemigrations --check` para comprobar que ningún cambio en los modelos quedó sin su migración. Las Figuras 13 y 14 son la evidencia de las migraciones aplicadas en MySQL.

Figura 13

Migraciones aplicadas y verificación de que el esquema está al día

```
terminal — docker compose exec api

$ python manage.py showmigrations
admin
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
authtoken
 [X] 0001_initial
 [X] 0002_auto_20160226_1747
 [X] 0003_tokenproxy
 [X] 0004_alter_tokenproxy_options
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
sessions
 [X] 0001_initial
votaciones
 [X] 0001_initial
 [X] 0002_pedir_documento
 [X] 0003_presentaciones_colores_respuestas_duelo

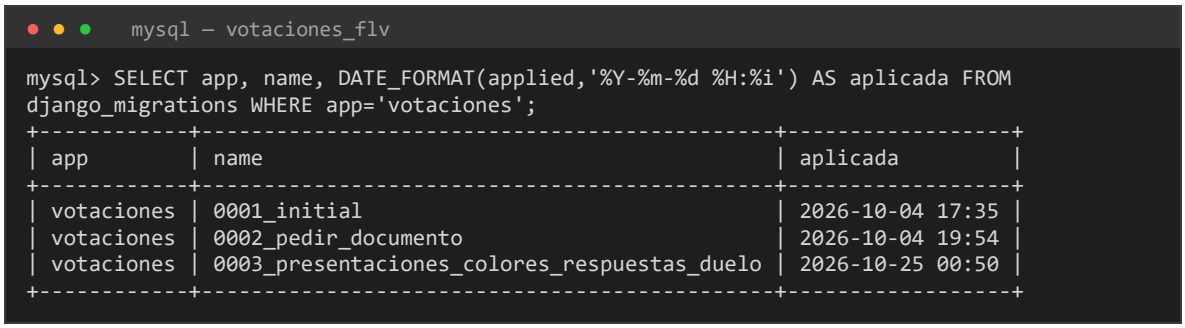
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, authtoken, contenttypes, sessions, votaciones
Running migrations:
  No migrations to apply.

$ python manage.py makemigrations --check --dry-run
No changes detected
```

Nota. Salida de consola tomada el 24 de octubre de 2026 en el entorno Docker local con MySQL 8.4. Todas las migraciones de Django y de la aplicación votaciones están aplicadas ([X]); migrate no tiene cambios pendientes y makemigrations --check confirma que los modelos coinciden con las migraciones.

Figura 14

Migraciones registradas por Django en la tabla django_migrations de MySQL



Nota. Consulta tomada el 24 de octubre de 2026 en el entorno Docker local con MySQL 8.4. Django guarda en MySQL qué migraciones se aplicaron y cuándo; así cada integrante obtiene el mismo esquema con python manage.py migrate.

Autenticación

La autenticación usa el sistema de usuarios de Django con un modelo propio, Usuario, que hereda de AbstractUser y se declara en AUTH_USER_MODEL. El correo electrónico reemplaza al nombre de usuario para iniciar sesión, las contraseñas se guardan con el hash PBKDF2 de Django y se validan con AUTH_PASSWORD_VALIDATORS. En el sitio, el registro usa un formulario de Django (RegistroForm) y el inicio y cierre de sesión usan authenticate(), login() y logout(), que guardan la sesión en una cookie protegida; el cierre de sesión solo se acepta por POST con token CSRF. El panel exige sesión de administrador mediante el decorador solo_administrador. La Figura 16 muestra en MySQL las contraseñas cifradas y las sesiones activas.

Figura 15

Vistas de registro, inicio y cierre de sesión del sitio

app/api/portal/vistas/cuentas.py

```python
def registro(request):
    form = RegistroForm(request.POST or None, pedir_documento=pedir_documento)
    if request.method == "POST" and form.is_valid():
        usuario = servicios.registrar_votante(**form.datos_cuenta())
        login(request, usuario, backend="django.contrib.auth.backends.ModelBackend")
        return redirect(destino_seguro(request, reverse("portal:categorias")))
    return render(request, "portal/registro.html", {"form": form, ...})

def iniciar_sesion(request):
    form = LoginForm(request.POST or None, request=request)   # usa authenticate()
    if request.method == "POST" and form.is_valid():
        login(request, form.usuario)
        return redirect(destino_seguro(request, por_defecto))
    return render(request, "portal/login.html", {"form": form, ...})

@require_POST
def cerrar_sesion(request):
    logout(request)
    return redirect("portal:inicio")
```

Nota. Fragmento del repositorio (app/api/portal/vistas/cuentas.py). Se omiten líneas que no son necesarias para la explicación.

Figura 16

Usuarios con contraseña cifrada y sesiones de Django guardadas en MySQL

mysql — votaciones_flv

```
mysql> SELECT rol, COUNT(*) AS usuarios, LEFT(MIN(password), 22) AS algoritmo_hash
       FROM usuario GROUP BY rol;
+---------------+----------+-----------------------+
| rol           | usuarios | algoritmo_hash        |
+---------------+----------+-----------------------+
| votante       |      103 | pbkdf2_sha256$1000000$ |
| administrador |        1 | pbkdf2_sha256$1000000$ |
+---------------+----------+-----------------------+

mysql> SELECT COUNT(*) AS sesiones_activas FROM django_session WHERE expire_date > NOW();
+------------------+
| sesiones_activas |
+------------------+
|               12 |
+------------------+
```

Nota. Consulta tomada el 24 de octubre de 2026 en el entorno Docker local con MySQL 8.4. Las contraseñas se guardan cifradas con PBKDF2-SHA256 (solo se muestra el prefijo del algoritmo, nunca la contraseña) y las sesiones activas quedan en la tabla django_session.

Administración: panel propio y Django Admin

Las ediciones, categorías, votaciones y opciones se administran en un panel propio construido con vistas, formularios y plantillas de Django (/panel), con el diseño de la marca: listados con búsqueda y filtros, formularios en ventana modal, activar o desactivar, publicar y cerrar votaciones (Figuras 18 a 33). Las reglas de negocio se aplican igual que en la API: no se publica una votación con menos de dos opciones, no se borra lo que tiene votos y cada cambio queda en la auditoría. Django Admin sigue disponible como complemento (Figuras 45 a 48).

Figura 17

Registro de los modelos en Django Admin

app/api/votaciones/votacion/admin.py

```python
class OpcionInline(admin.TabularInline):
    model = Opcion
    extra = 0


@admin.register(Edicion)
class EdicionAdmin(TrazableAdmin):
    list_display = ["nombre", "anio", "fecha_inicio", "fecha_fin", "estado"]


@admin.register(Categoria)
class CategoriaAdmin(TrazableAdmin):
    list_display = ["nombre", "edicion", "activa", "orden"]
    list_filter = ["edicion", "activa"]


@admin.register(Votacion)
class VotacionAdmin(TrazableAdmin):
    list_display = ["titulo", "categoria", "fecha_apertura", "fecha_cierre", "publicada",
"estado"]
    list_filter = ["categoria__edicion", "categoria", "publicada"]
    inlines = [OpcionInline]                        # opciones dentro de la votación


@admin.register(Voto)
class VotoAdmin(admin.ModelAdmin):
    def has_change_permission(self, request, obj=None):
        return False                                # los votos no se modifican
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/admin.py). Se omiten líneas que no son necesarias para la explicación.

Capturas de pantalla comentadas

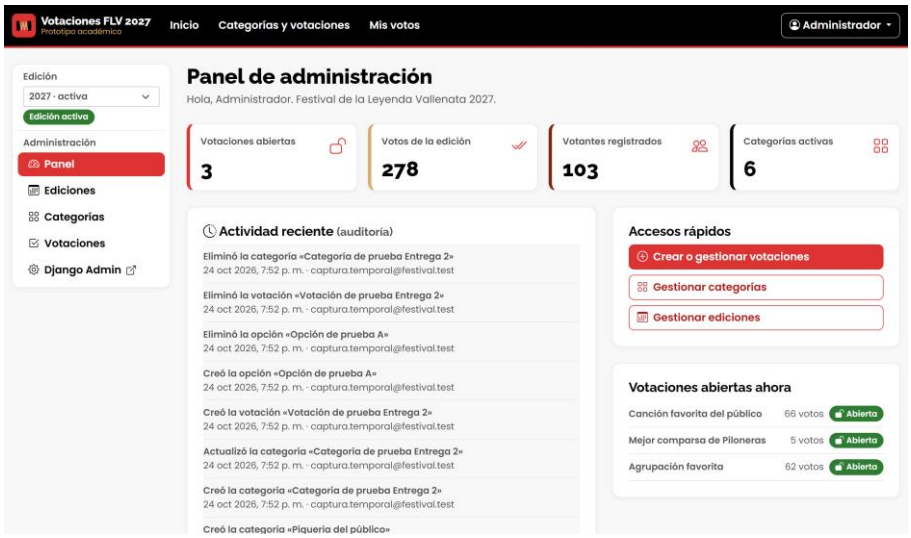
Las capturas del panel propio se tomaron el 24 de octubre de 2026; las del sitio, el 20 de octubre, y las de Django Admin y la API, el 18 de octubre de 2026, en el entorno Docker local (MySQL 8.4 y Django), con datos de prueba ficticios. Se usaron un administrador, un votante y registros temporales que se borraron al terminar; ninguna captura muestra contraseñas.

Panel de administración propio (CRUD con Django)

El panel propio (/panel) es la herramienta de administración del sistema. Las figuras siguen un ciclo completo sobre MySQL: crear, consultar, editar y eliminar una categoría, y crear una votación con su opción participante.

Figura 18

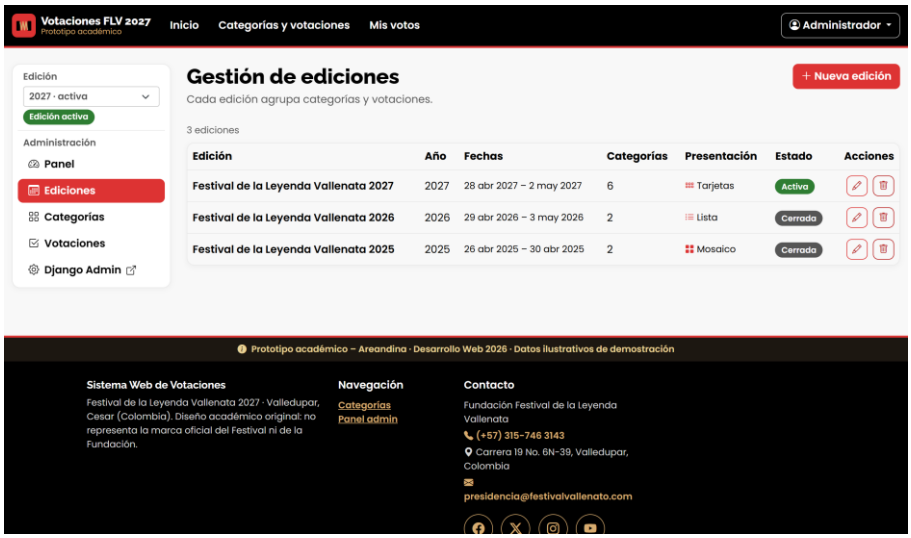
Inicio del panel de administración propio



Nota. Indicadores de la edición, actividad reciente (auditoría), accesos rápidos y votaciones abiertas. Solo entra un usuario con rol de administrador; el menú lateral lleva a ediciones, categorías y votaciones.

Figura 19

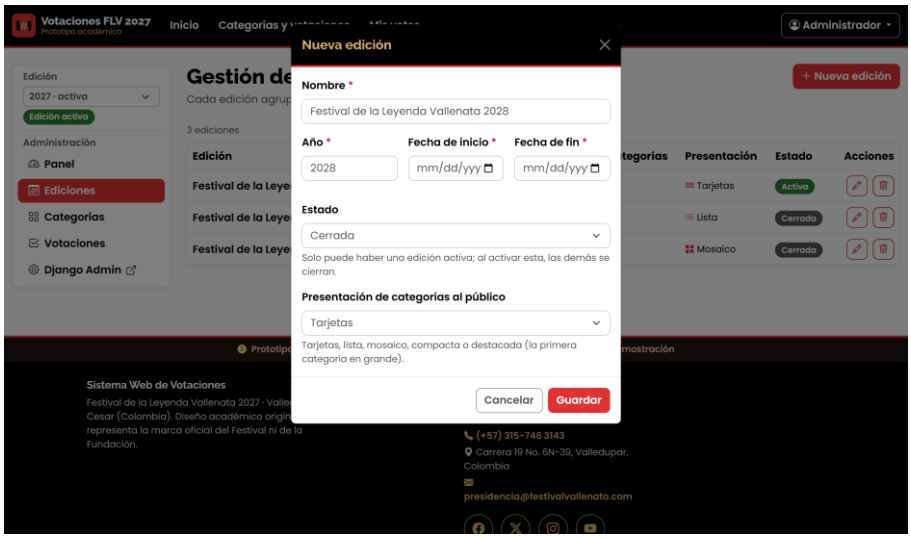
Gestión de ediciones



Nota. Listado de las ediciones guardadas en MySQL con sus fechas, número de categorías, presentación y estado; desde aquí se crean, editan y eliminan.

Figura 20

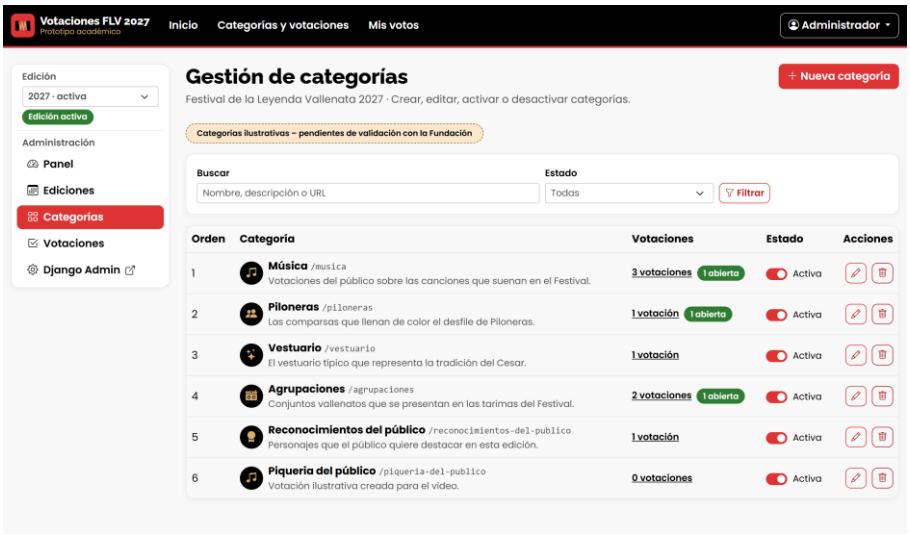
Formulario para crear una edición



Nota. Ventana modal con nombre, año, fechas, estado y presentación de las categorías. Solo puede haber una edición activa: al activar una, las demás se cierran.

Figura 21

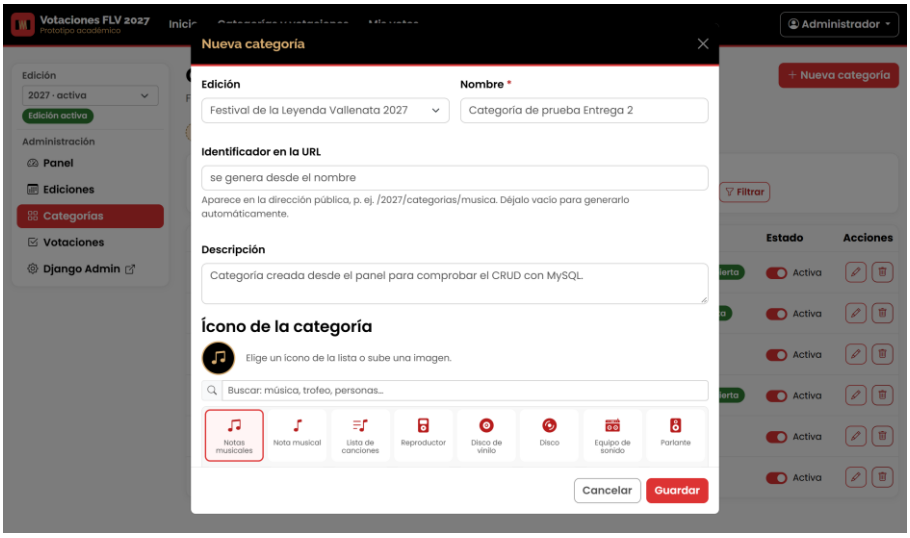
Gestión de categorías de la edición 2027



Nota. Cada categoría muestra su ruta pública, sus votaciones y un interruptor para activarla o desactivarla. El selector de la izquierda cambia la edición consultada.

Figura 22

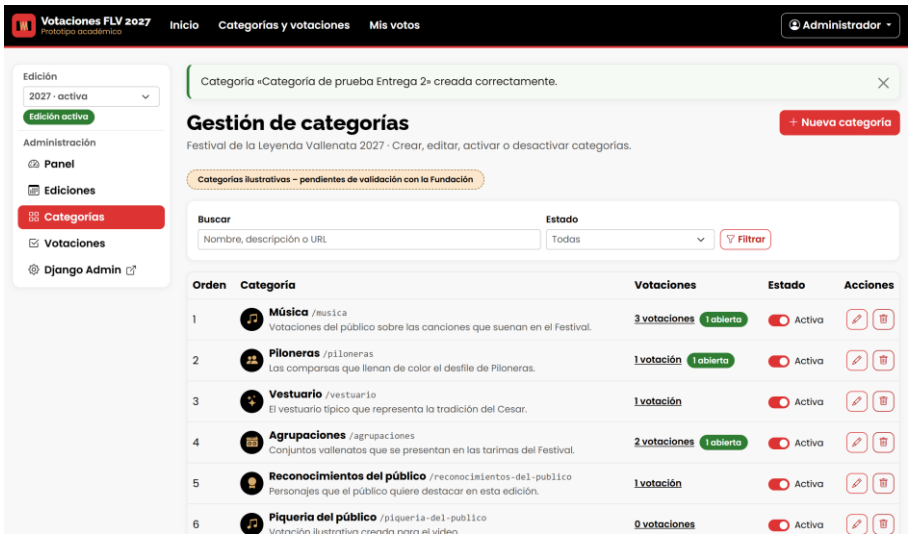
Creación de una categoría desde el panel



Nota. Formulario de Django en ventana modal: edición, nombre, identificador en la URL, descripción, ícono y orden. Se registra «Categoría de prueba Entrega 2».

Figura 23

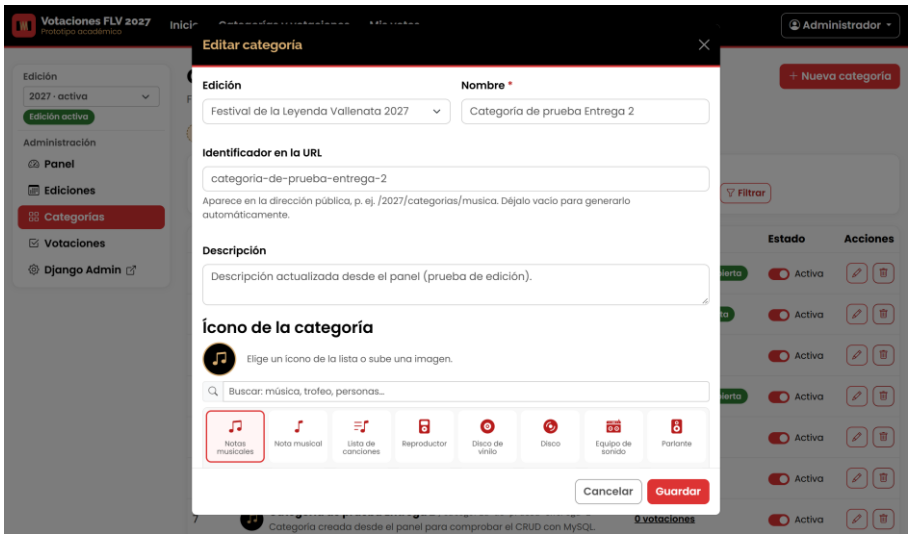
Confirmación de la categoría creada



Nota. El mensaje confirma que el registro se guardó en MySQL y la categoría aparece al final del listado; la acción queda en la auditoría.

Figura 24

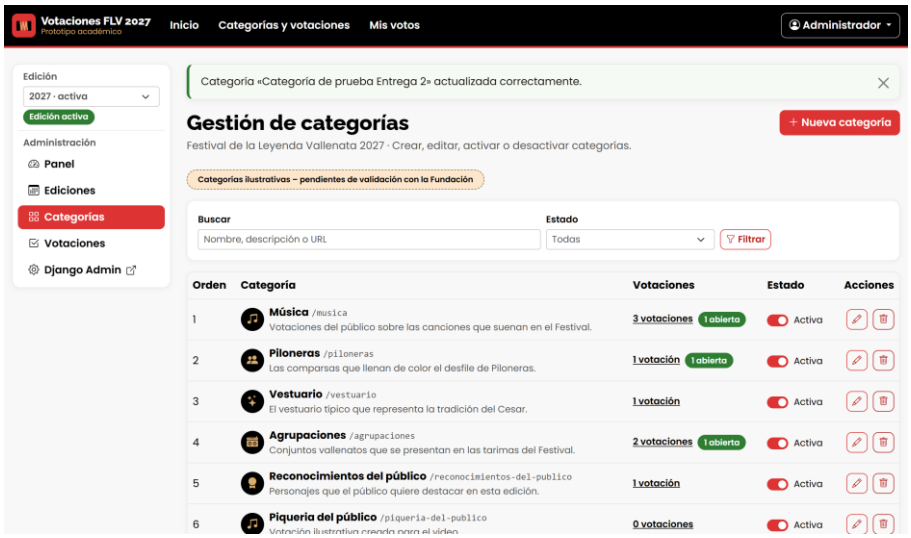
Edición de la categoría



Nota. El mismo formulario se abre con los datos guardados; se cambia la descripción de la categoría de prueba.

Figura 25

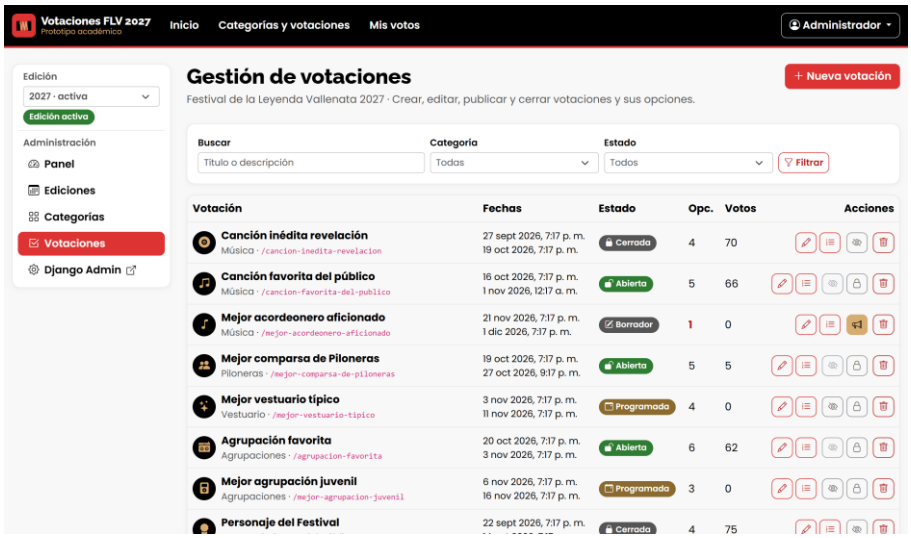
Confirmación de la categoría actualizada



Nota. El panel confirma la actualización y el listado muestra la nueva descripción leída de MySQL.

Figura 26

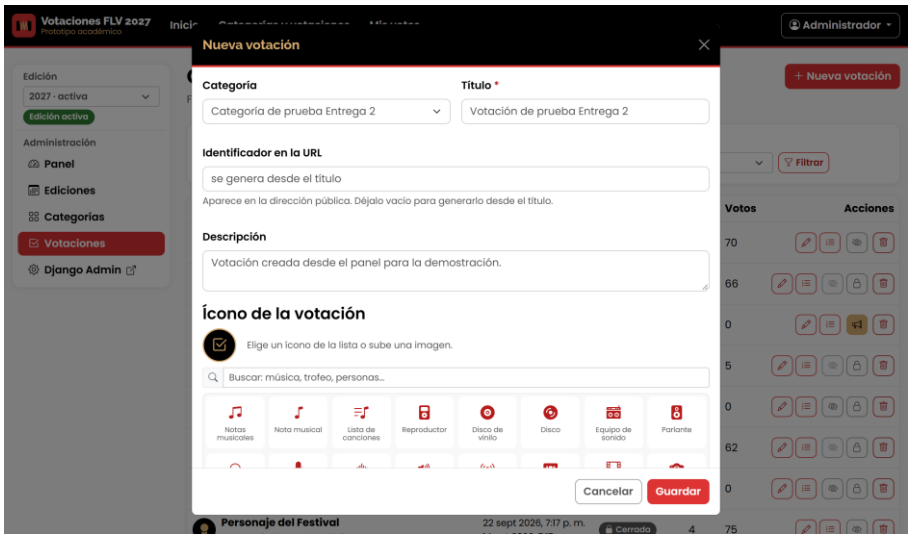
Gestión de votaciones



Nota. Cada votación muestra su categoría, fechas, estado, número de opciones y votos, y las acciones editar, opciones, publicar o despublicar, cerrar y eliminar.

Figura 27

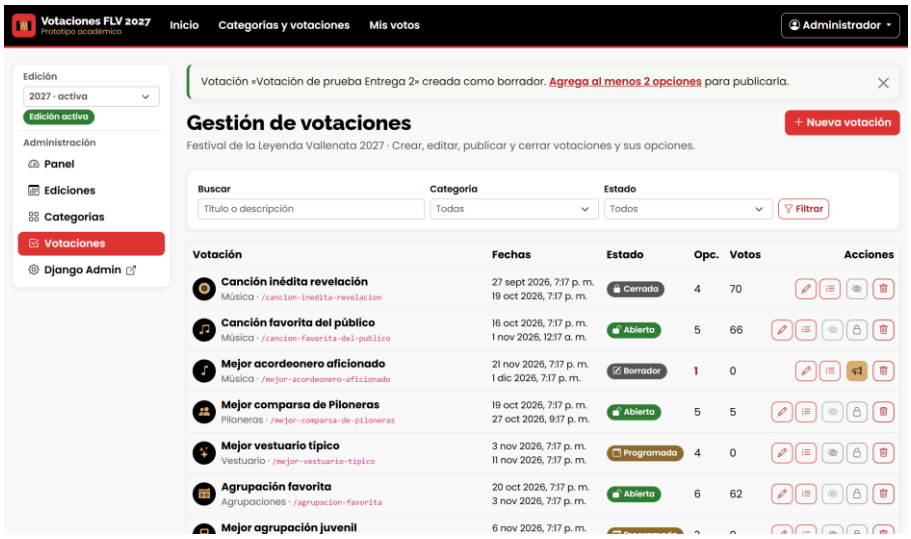
Creación de una votación en la categoría nueva



Nota. Se elige la categoría creada y se registran el título, la descripción, el ícono, las fechas de apertura y cierre, los votos por usuario y la forma de presentar las opciones.

Figura 28

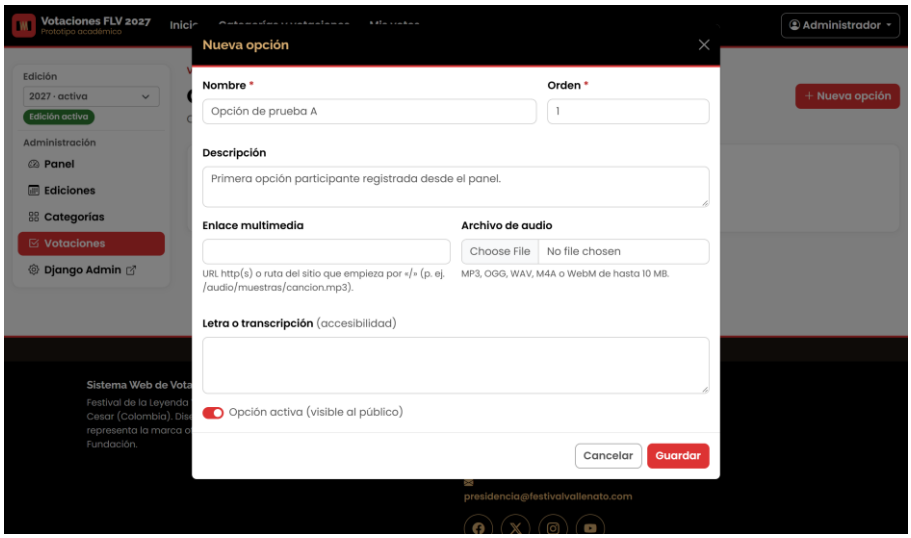
Votación guardada como borrador



Nota. La votación queda en borrador y el panel avisa que necesita al menos dos opciones para publicarse, regla de negocio que también aplica la API.

Figura 29

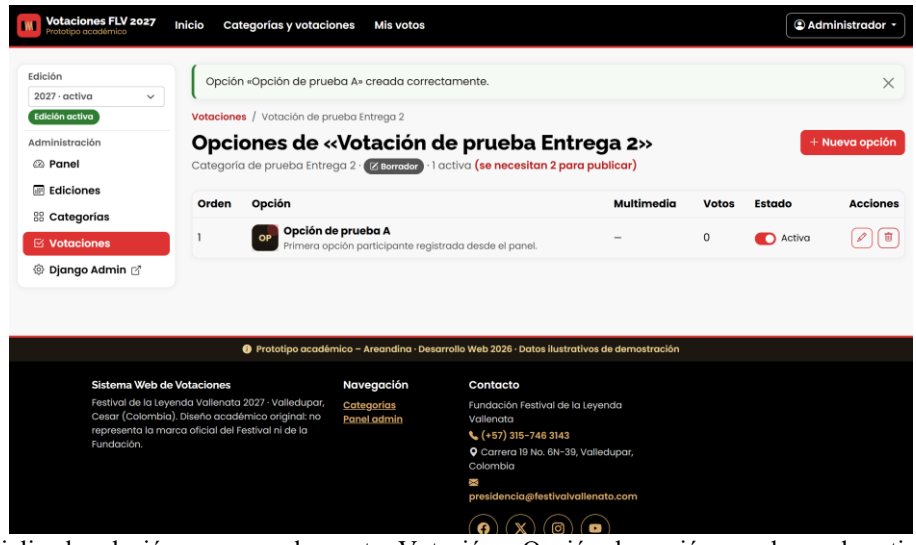
Creación de una opción participante



Nota. Formulario con nombre, orden, descripción, enlace o archivo de audio y la letra o transcripción para accesibilidad.

Figura 30

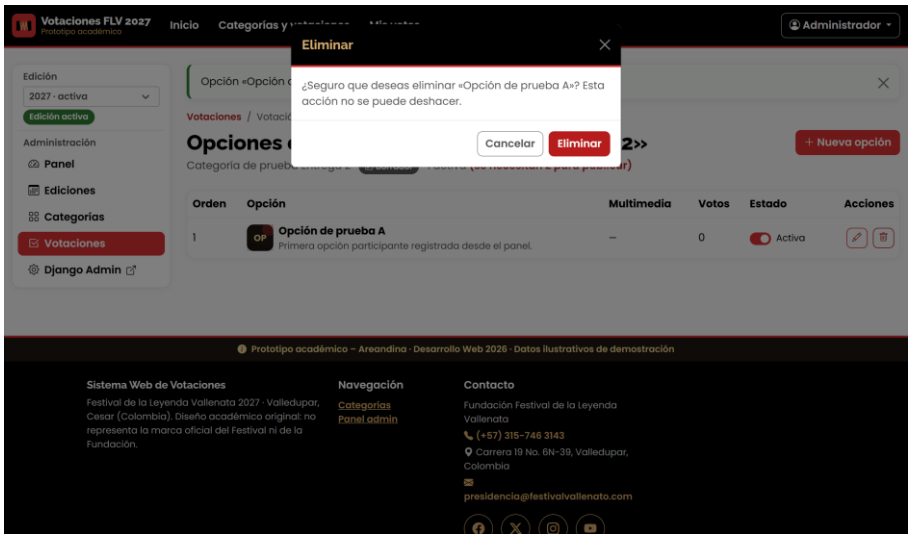
Opciones de la votación



Nota. Materializa la relación uno a muchos entre Votación y Opción: la opción creada queda activa dentro de la votación de prueba.

Figura 31

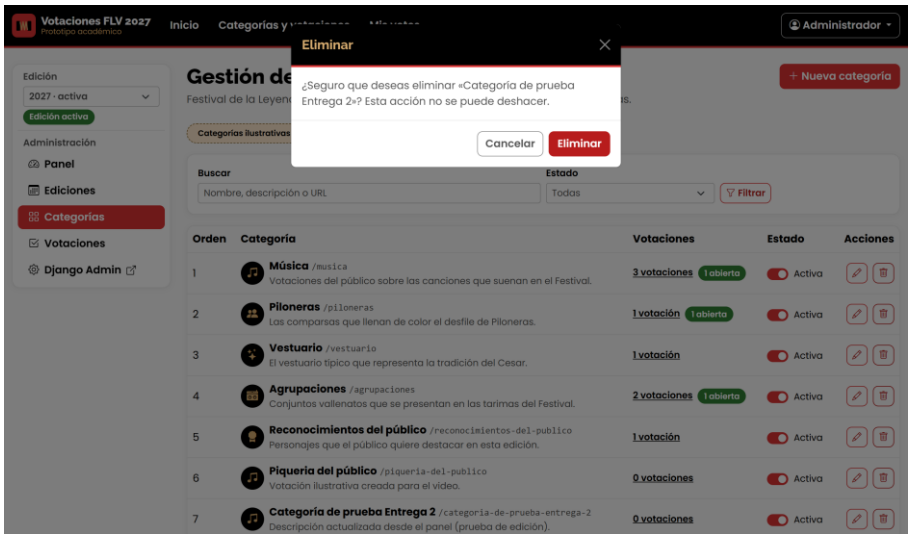
Confirmación antes de eliminar una opción



Nota. Toda eliminación pide confirmación en una ventana modal y se envía por POST con token CSRF.

Figura 32

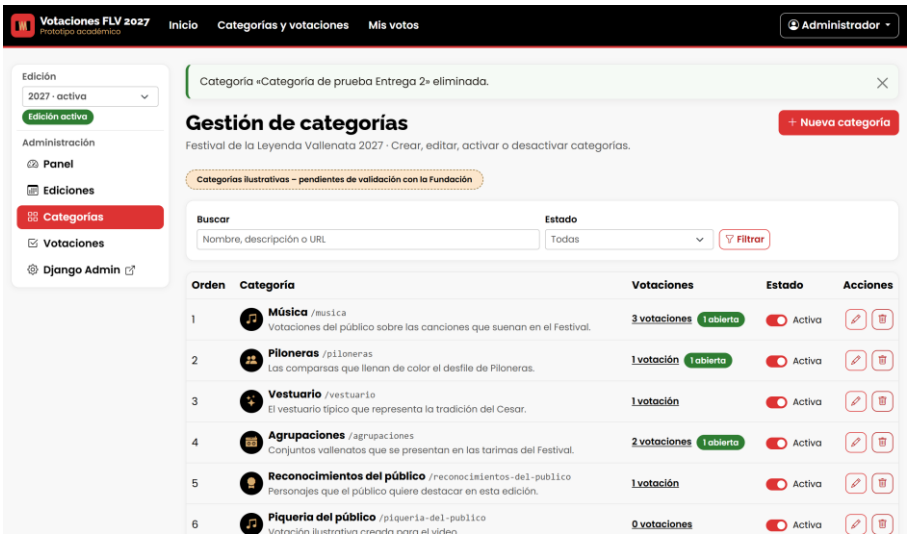
Eliminación de la categoría de prueba



Nota. Antes se eliminaron su opción y su votación, porque el panel no permite borrar una categoría que tenga votaciones asociadas.

Figura 33

Confirmación de la categoría eliminada

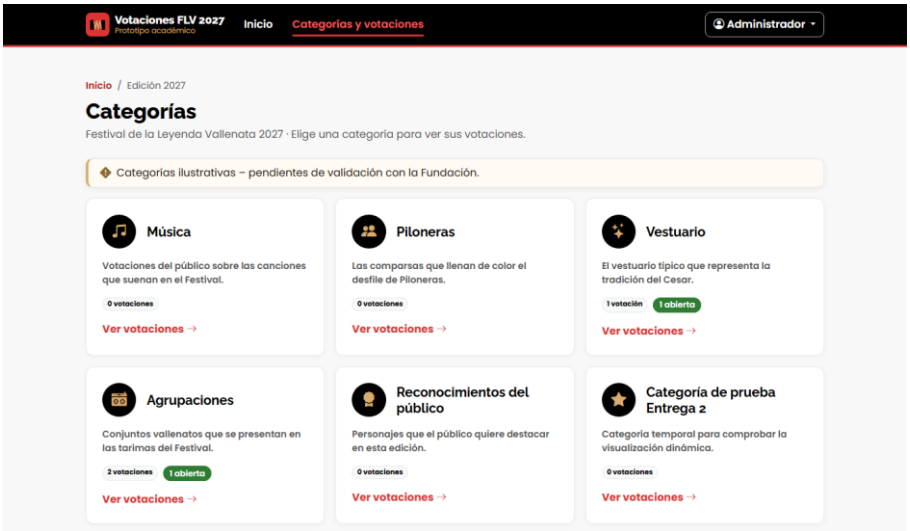


Nota. El mensaje confirma la eliminación y el registro ya no aparece en el listado ni en el sitio público.

Visualización dinámica

Figura 34

La categoría creada en el panel aparece en el sitio

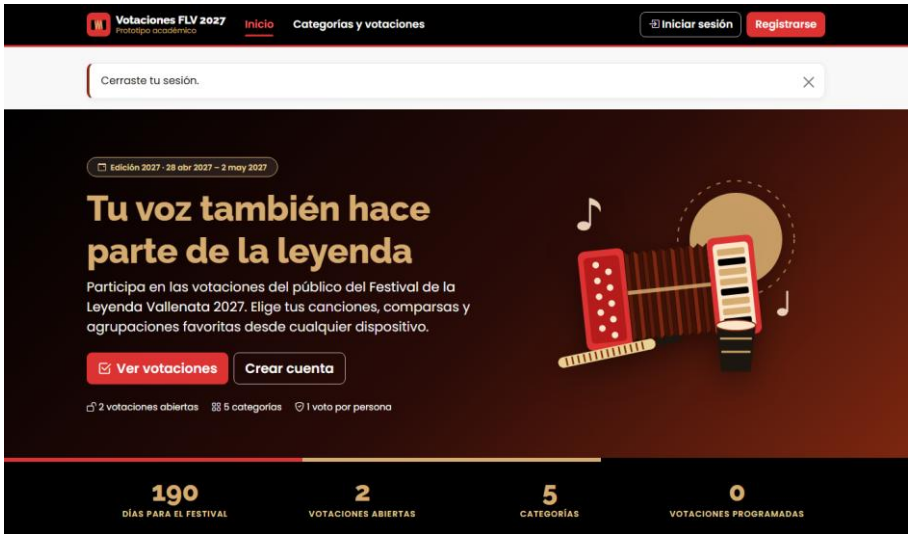


Nota. Demuestra que la interfaz pública no tiene datos fijos: la categoría registrada desde el panel (Figura 22) se muestra de inmediato porque la vista la consulta en MySQL. Después de la prueba se borró.

Interfaz pública con datos de MySQL

Figura 35

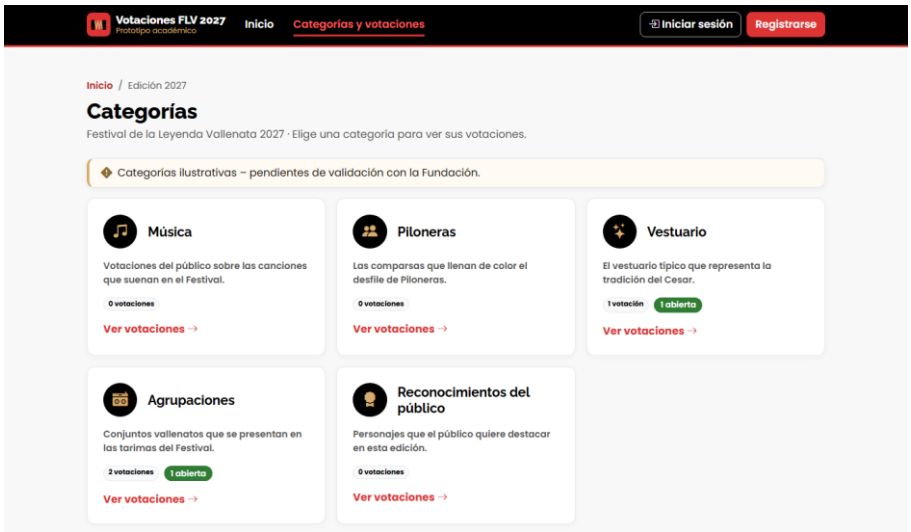
Página de inicio



Nota. La vista inicio (portal/vistas/publico.py) consulta la edición activa y sus votaciones abiertas con el ORM y la plantilla inicio.html las muestra.

Figura 36

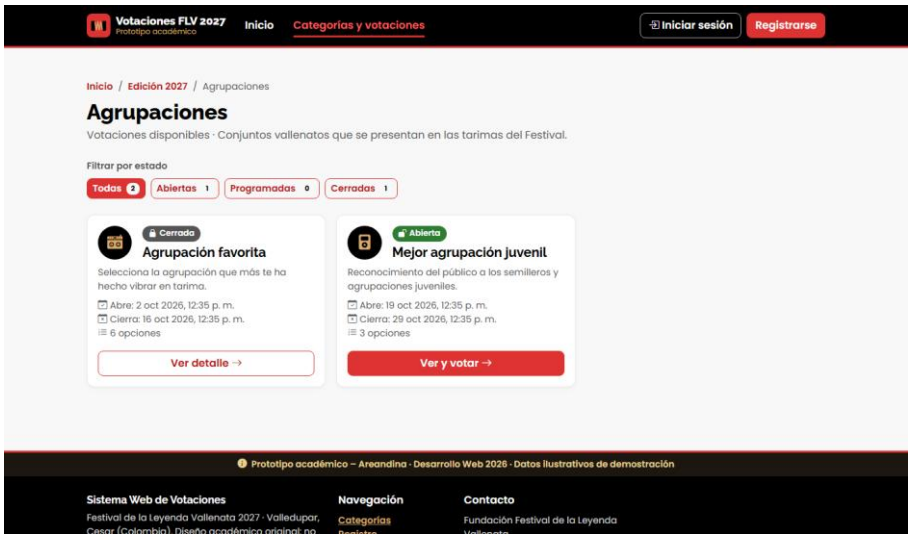
Categorías de la edición 2027



Nota. La plantilla recorre las categorías activas de la edición; el número de votaciones de cada una lo calcula la vista con una consulta a MySQL.

Figura 37

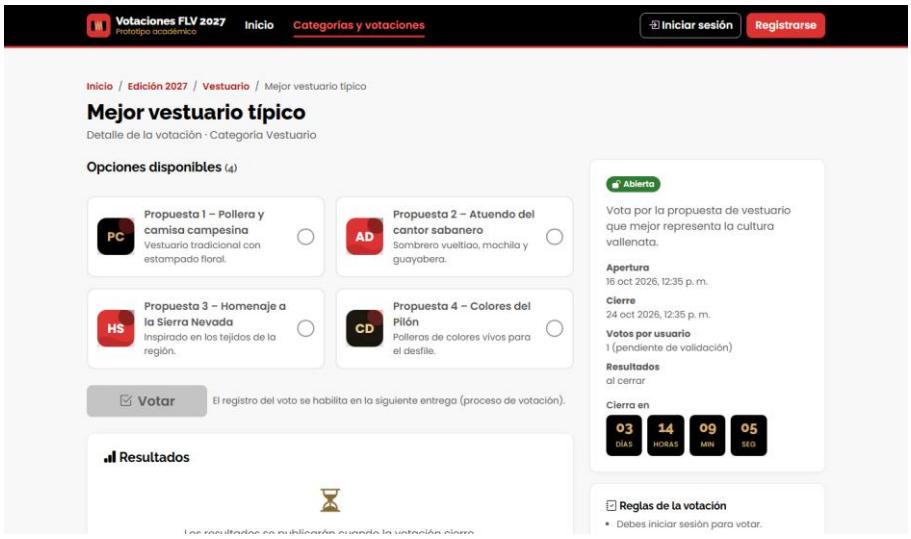
Votaciones de una categoría



Nota. Al elegir una categoría se listan sus votaciones con su estado y sus fechas; el filtro por estado usa parámetros de la URL (?estado=abierta).

Figura 38

Detalle de una votación con sus opciones

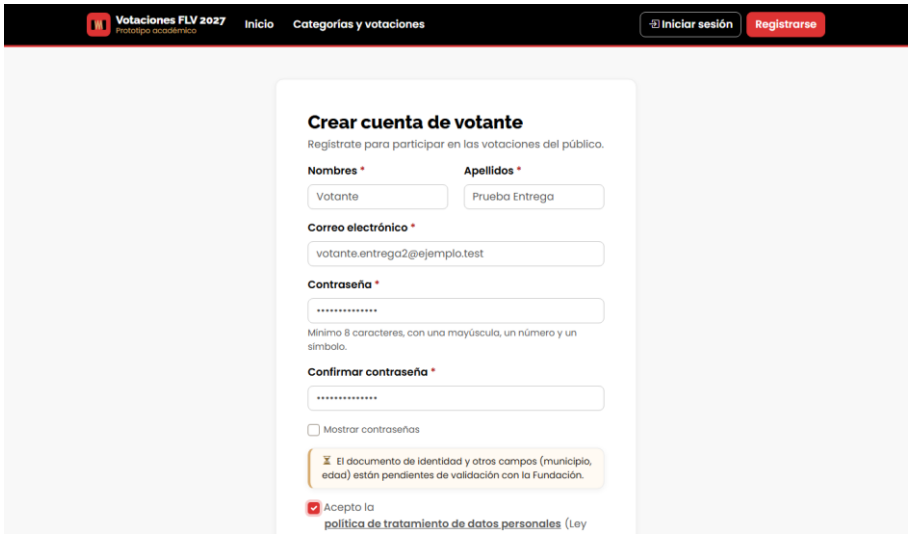


Nota. Muestra la votación y las opciones registradas en el panel. El registro definitivo del voto corresponde a la Entrega 3, por eso el botón «Votar» aparece deshabilitado.

Autenticación

Figura 39

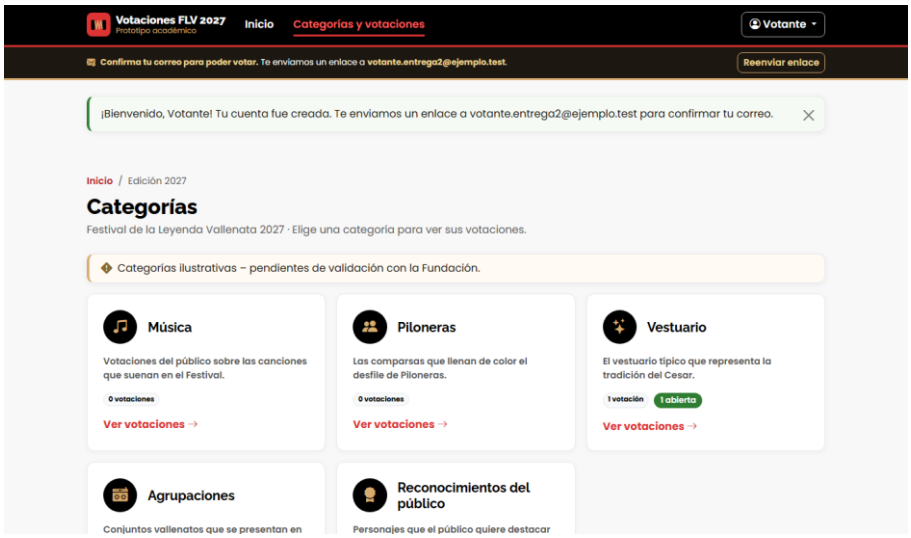
Formulario de registro de un votante



Nota. Formulario de Django (RegistroForm): nombres, apellidos, correo, contraseña y autorización de tratamiento de datos (Ley 1581 de 2012). Las contraseñas se ven ocultas.

Figura 40

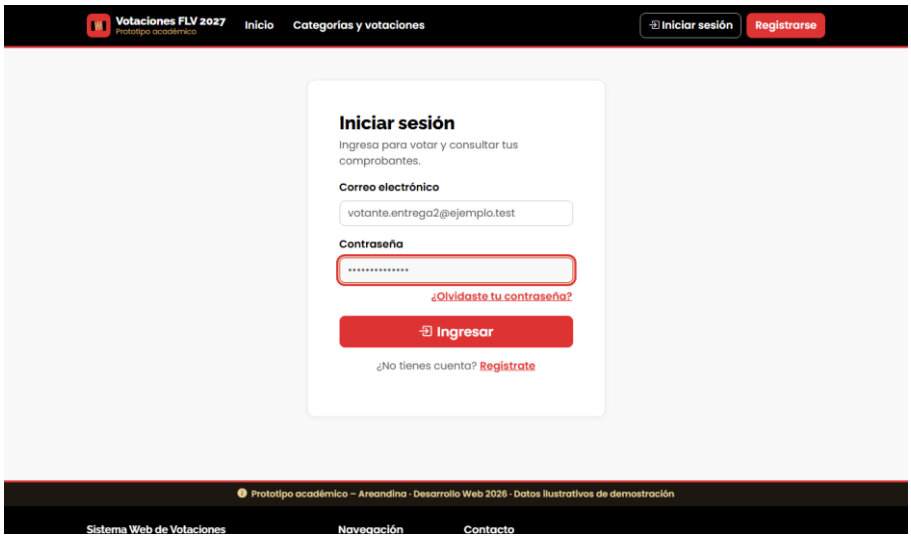
Resultado del registro



Nota. La cuenta queda creada y la sesión iniciada con login(); el mensaje confirma el registro y la barra superior pide confirmar el correo antes de votar (RN-18 de la Entrega 1).

Figura 41

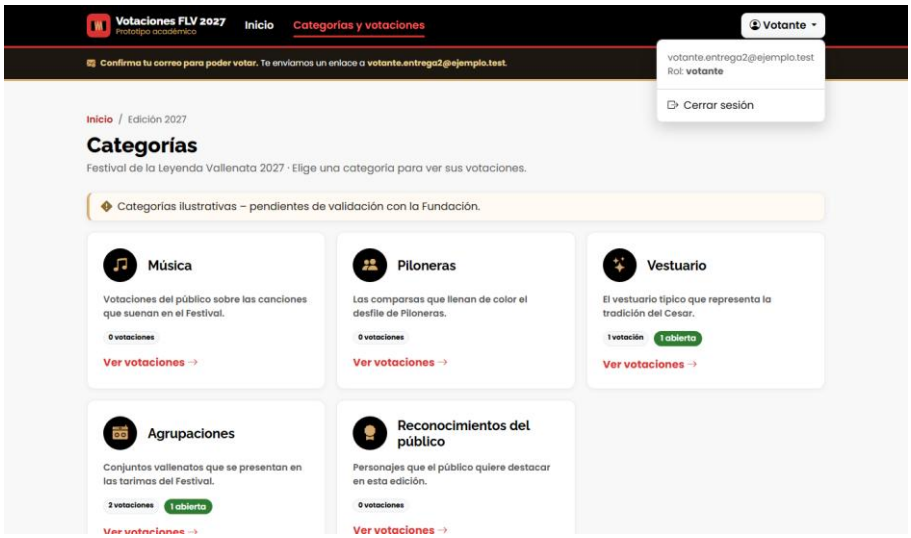
Inicio de sesión



Nota. Inicio de sesión con correo y contraseña de un votante de prueba; la vista valida con authenticate().

Figura 42

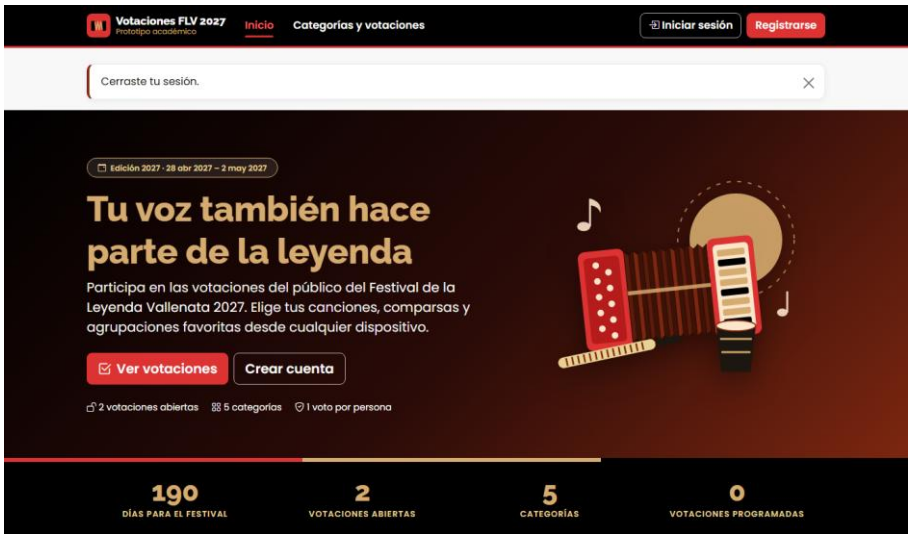
Sesión iniciada



Nota. Con la sesión de Django activa, la barra superior muestra el nombre del usuario y el menú con «Cerrar sesión».

Figura 43

Sesión cerrada

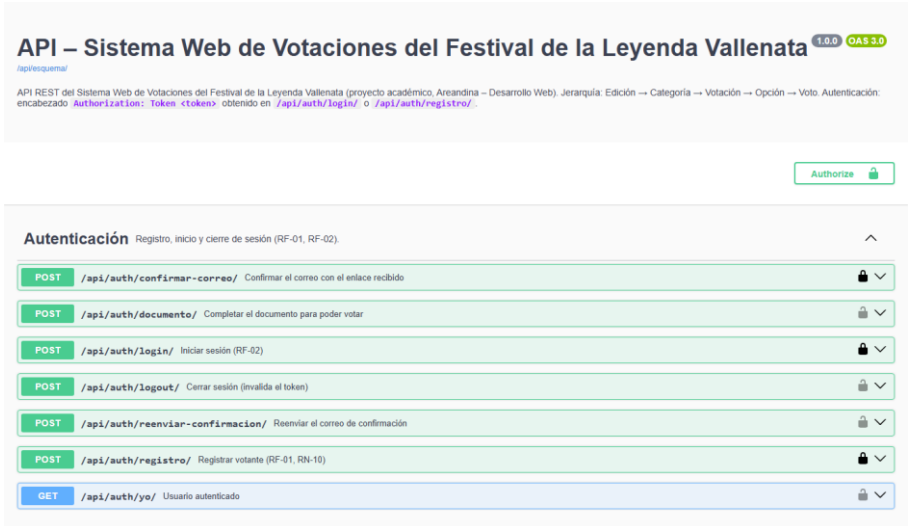


Nota. El cierre de sesión se envía por POST con token CSRF; logout() elimina la sesión y vuelven los botones «Iniciar sesión» y «Registrarse».

Vistas y documentación de la API

Figura 44

Documentación de las vistas de la API (Swagger)



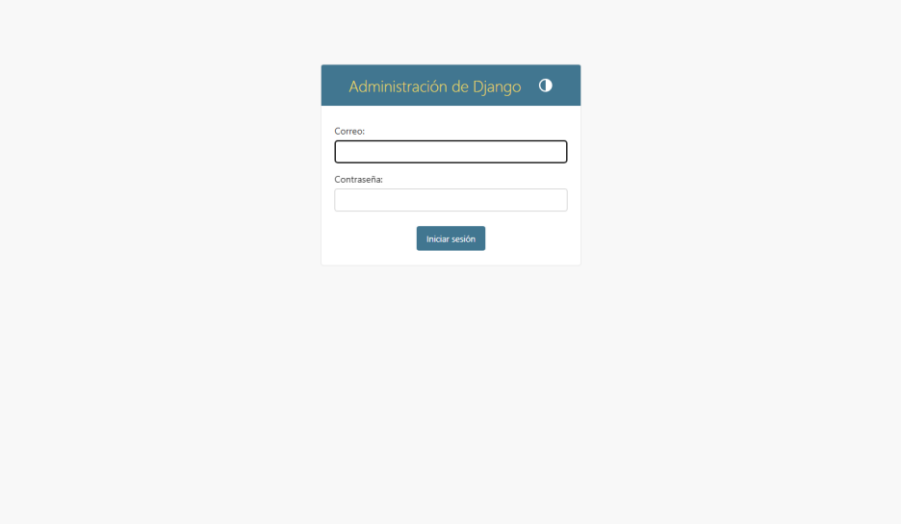
Nota. Las vistas de Django quedan documentadas en /api/docs/ con el esquema OpenAPI generado automáticamente; se ven los grupos de autenticación, consulta pública y administración.

Django Admin (complementario)

Django Admin se conserva como complemento para el equipo técnico: permite revisar los modelos registrados y los usuarios, pero la gestión diaria se hace en el panel propio.

Figura 45

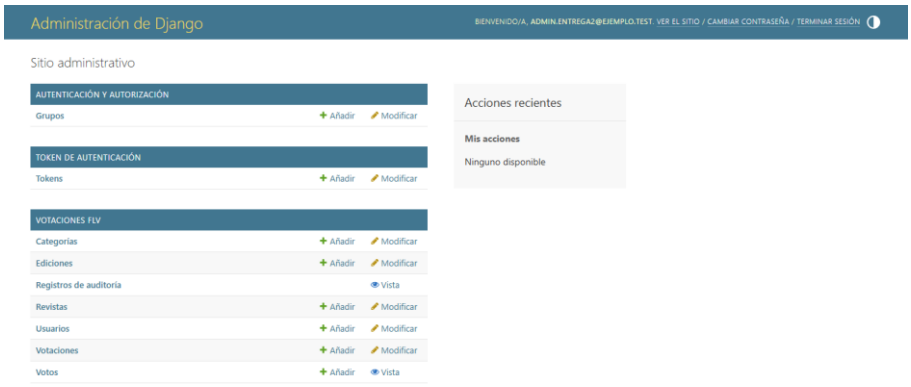
Inicio de sesión en Django Admin



Nota. Solo los usuarios con permiso de personal (is_staff) pueden entrar. El campo de usuario es el correo electrónico, porque el modelo Usuario lo define como USERNAME_FIELD.

Figura 46

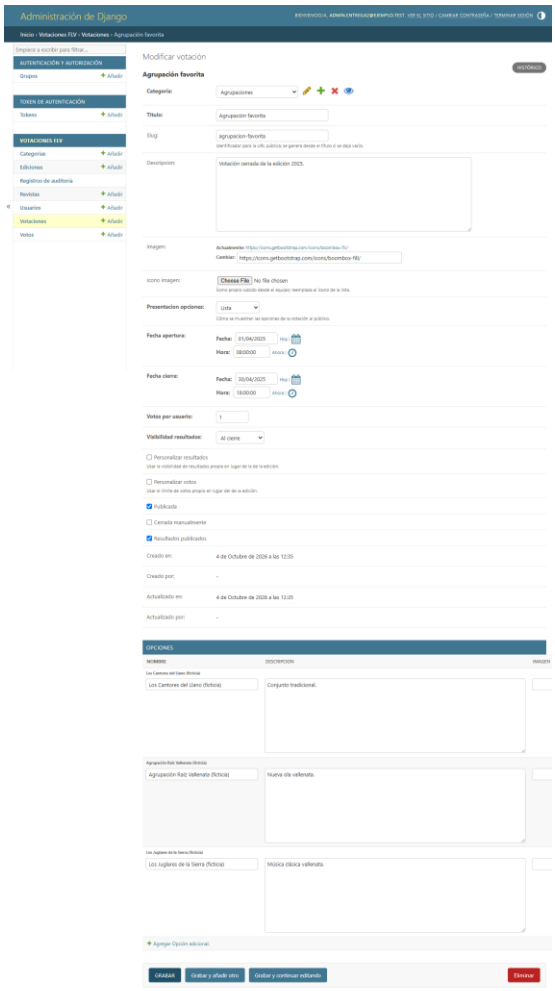
Modelos registrados en Django Admin



Nota. Demuestra que los modelos de la aplicación (Ediciones, Categorías, Votaciones, Usuarios, Votos, Revistas y Registros de auditoría) quedaron disponibles para administrarse. Votos y Auditoría solo permiten consulta.

Figura 47

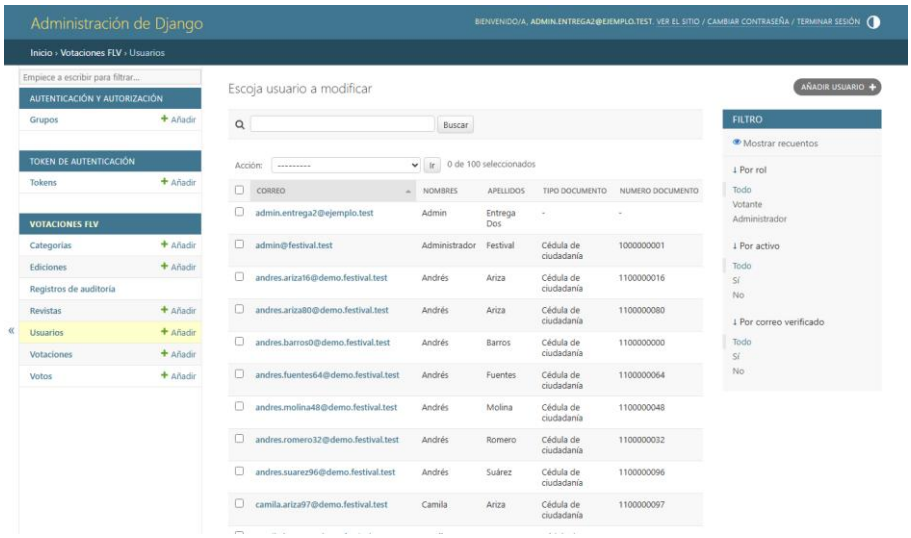
Edición de una votación con sus opciones en línea



Nota. Desde la misma pantalla se configura la votación (fechas, límite de votos, visibilidad de resultados) y se administran sus opciones participantes, lo que materializa la relación uno a muchos entre Votación y Opción. La captura se recortó para mostrar solo el nombre y la descripción de cada opción.

Figura 48

Usuarios registrados



Nota. Listado del modelo de usuario propio. Los datos son ficticios y las contraseñas se guardan cifradas (hash), por lo que nunca se muestran.

Pruebas y resultados

Pruebas automatizadas

El proyecto tiene 219 pruebas automatizadas escritas con el módulo de pruebas de Django: 180 de la API y de las reglas de negocio y 39 del sitio construido con vistas y plantillas (páginas públicas, registro, sesión y CRUD del panel). Cada prueba crea una base de datos temporal, ejecuta el caso y la destruye al final, por lo que no afecta los datos reales. Las pruebas se ejecutan en cada cambio que se sube a GitHub y, para esta entrega, también contra MySQL 8.4.

Procedimiento: docker compose exec api python manage.py test

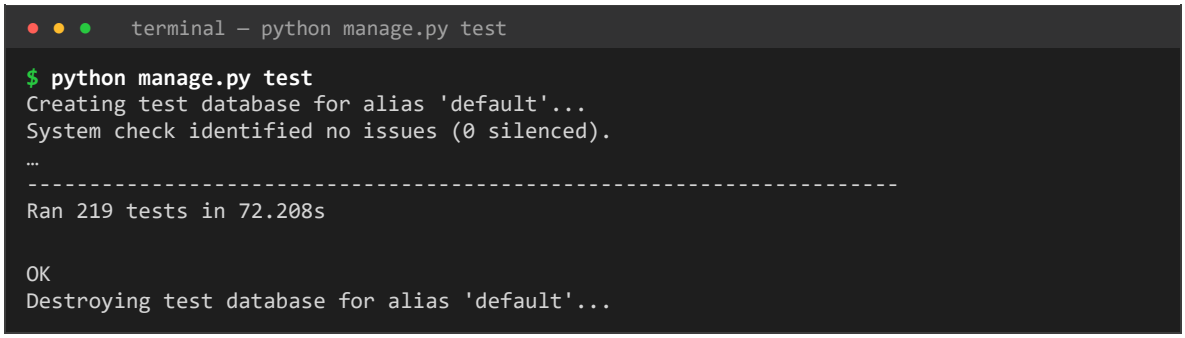
Tabla 8

Pruebas automatizadas por archivo

Archivo	Qué verifica	Pruebas
test_auth.py	Registro, inicio y cierre de sesión por la API, confirmación del correo, perfil	22
test_admin.py	Administración de ediciones, categorías, votaciones y opciones por la API; permisos	18
test_publico.py (API)	Consulta pública de ediciones, categorías, votaciones y opciones	14
test_integracion_web.py	Respuestas que consume la web React (formato y campos)	28
test_configuracion_edicion.py · test_votar.py	Configuración por edición y base del proceso de voto	17
test_participacion_integridad.py · test_seguridad.py	Reportes de integridad, permisos y cabeceras de seguridad	23
test_autoria.py · test_correo.py	Auditoría automática y correos de confirmación y comprobante	32
test_sitio.py, test_seo.py, test_marca.py, test_ruta_gestion.py	Contenido del sitio, mapa del sitio, marca y rutas	26
portal/tests/test_publico.py	Vistas y plantillas públicas: edición activa, categorías, filtros, detalle, borradores y resultados	12
portal/tests/test_cuentas.py	Registro con formulario de Django, inicio y cierre de sesión, redirecciones seguras, confirmación del correo	10
portal/tests/test_panel.py	Acceso al panel y CRUD de ediciones, categorías, votaciones y opciones con sus reglas	17
Total		219

Figura 49

Resultado de las pruebas contra MySQL



Nota. Salida de consola tomada el 20 de octubre de 2026 en Docker con MySQL 8.4: 219 pruebas ejecutadas en 72 segundos, todas correctas (OK).

Pruebas funcionales

Tabla 9

Pruebas funcionales realizadas

Funcionalidad	Procedimiento	Resultado esperado	Resultado
Conexión con MySQL	Arrancar Docker y ejecutar showmigrations	Migraciones aplicadas en MySQL 8.4 (Docker local)	Correcto (Figuras 13 y 14)
Ingreso a Django Admin	Entrar con un superusuario de prueba	Se ven los modelos registrados	Correcto (Figura 46)
Ingreso al panel propio	Iniciar sesión como administrador	Se abre /panel con los indicadores	Correcto (Figura 18)
Crear categoría	Registrar una categoría en la edición 2027 desde el panel	Se guarda y aparece en el listado	Correcto (Figuras 22–23)
Editar categoría	Cambiar la descripción de la categoría creada	Se actualiza en MySQL y en el listado	Correcto (Figuras 24–25)
Crear votación	Registrar una votación en la categoría nueva	Se guarda como borrador hasta tener dos opciones	Correcto (Figuras 27–28)
Opciones de una votación	Crear una opción y luego eliminarla	Se guarda y se elimina tras confirmar	Correcto (Figuras 29–31)
Eliminar categoría	Borrar la categoría de prueba sin votaciones	Desaparece del panel y del sitio	Correcto (Figuras 32–33)
Visualización dinámica	Abrir el sitio después de crear la categoría	El sitio la muestra sin cambiar código	Correcto (Figura 34)
Registro	Llenar el formulario de registro	Cuenta creada y aviso de confirmar correo	Correcto (Figuras 39–40)
Inicio de sesión	Ingresar correo y contraseña válidos	Se muestra el usuario en la barra	Correcto (Figuras 41–42)
Cierre de sesión	Elegir «Cerrar sesión»	Vuelven los botones de ingreso	Correcto (Figura 43)

Repositorio

El código fuente está en un repositorio público de GitHub. El archivo README explica la estructura del proyecto y los pasos de instalación y ejecución con Docker y MySQL: clonar el repositorio, ejecutar docker compose up -d --build, cargar los datos de prueba con python manage.py cargar_demo y crear un superusuario con python manage.py createsuperuser. El sitio Django queda en http://localhost:8096/ y el panel en /panel.

Repositorio: https://github.com/juandiegows/votaciones-festival-vallenato

Instalación y ejecución: https://github.com/juandiegows/votaciones-festival-vallenato#instalación-y-ejecución-django--mysql

Versión publicada: https://votaciones.juandiegows.com

Demostración sin servidor (GitHub Pages): https://juandiegows.github.io/votaciones-festival-vallenato/

Video demostrativo

El video muestra la estructura del proyecto Django, la conexión con MySQL por medio del adaptador, los modelos y las migraciones, el CRUD de ediciones, categorías, votaciones y opciones en el panel, la visualización dinámica de los datos en el sitio, el registro, el inicio y el cierre de sesión, y la ejecución de las pruebas. Cada integrante presenta una parte.

Enlace al video: https://youtu.be/4k3m2sHrQvM

Conclusiones y distribución del trabajo

Aprendizajes

- Patrón MVT. Separar modelos, vistas y plantillas permitió construir el sitio y el panel con el mismo diseño de la marca y probar cada vista de forma independiente con el cliente de pruebas de Django.
- Integridad y reglas en un solo lugar. Las restricciones de la base de datos y la capa de servicios hacen que el panel, Django Admin y la API apliquen las mismas reglas (por ejemplo, no publicar una votación con menos de dos opciones).

• Adaptadores. La base de datos (MySQL o SQLite) y el correo (ZeptoMail o SMTP) se cambian con una variable de entorno, sin tocar el código, y ninguna categoría ni votación está escrita en el código.

Dificultades

• **Migraciones.** Al empezar se unieron varias migraciones en una sola inicial para que MySQL y SQLite partieran del mismo esquema.

• **Permisos de MySQL.** Las pruebas contra MySQL requieren que el usuario de la base tenga permiso para crear la base de pruebas; quedó documentado en el README.

• **Requerimientos del cliente.** Las categorías y reglas definitivas siguen pendientes de validación con la Fundación, por lo que los datos de prueba son ilustrativos.

Pendientes para la siguiente entrega

• Registro definitivo del voto y control de votos duplicados.

• Reglas completas de elegibilidad y verificación del votante.

• Publicación final de resultados, auditoría avanzada y pruebas de carga.

• Respuestas de la Fundación a las preguntas P-01 a P-17 de la Entrega 1.

Distribución del trabajo

Tabla 10

Responsabilidades de cada integrante en la Entrega 2

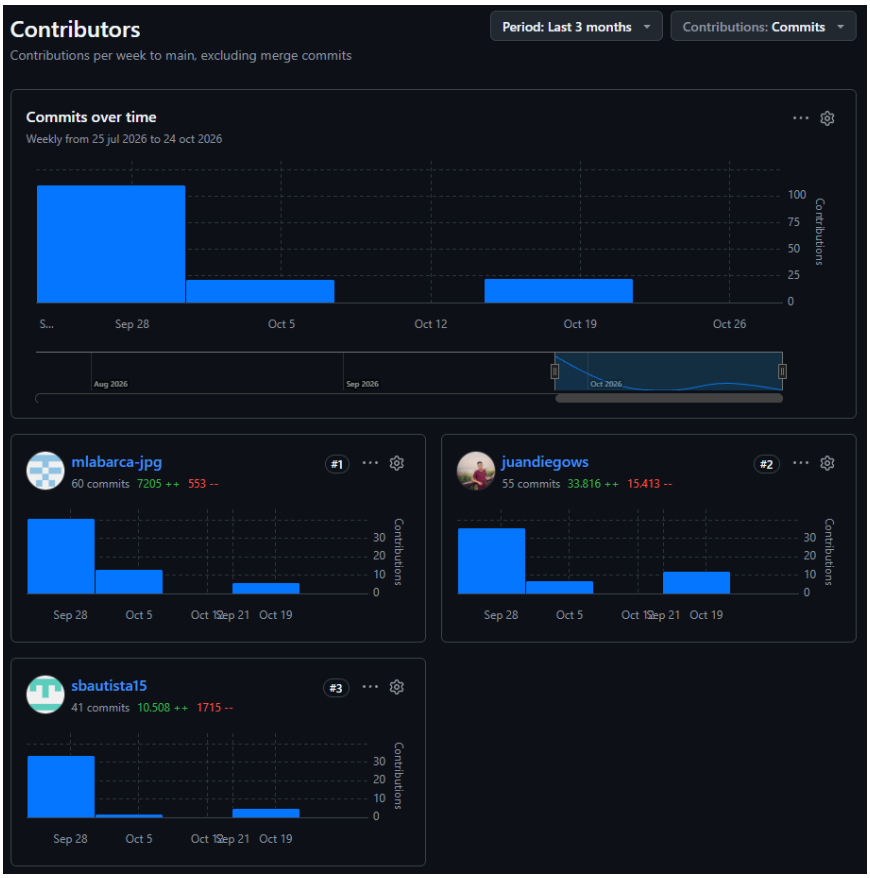
Integrante	Responsabilidades
Juan Mejía Maestre	Coordinación del equipo; app portal: estilos de la marca, vistas y plantillas públicas, registro e inicio de sesión; Docker y revisión de coherencia.
Sebastián Bautista Martínez	Proyecto Django y adaptador de MySQL; modelos, relaciones y migraciones; panel de administración (CRUD de ediciones, categorías, votaciones y opciones).
María Labarca Briceño	Formularios de registro, sesión y administración; pruebas automatizadas del sitio; documentación de la arquitectura, capturas e informe.

Nota. Elaboración propia a partir de los aportes registrados en el historial del repositorio de GitHub. Todos los integrantes revisaron el documento completo y pueden explicar cualquier parte de la solución.

La participación de los tres integrantes se puede verificar en el repositorio. La Figura 50 muestra la página de contribuidores de GitHub (Insights → Contributors) en los últimos tres meses: María Labarca Briceño (mlabarca-jpg) registra 60 commits, Juan Mejía Maestre (juandiegows) 55 y Sebastián Bautista Martínez (sbautista15) 41, con aportes en cada semana de trabajo.

Figura 50

Aportes de cada integrante en el repositorio de GitHub



Nota. Captura de GitHub tomada el 25 de octubre de 2026 (commits en la rama main, sin contar fusiones). Cada integrante trabaja con su propia cuenta, por lo que el historial del repositorio respalda la distribución de la Tabla 10.

AREANDINA

Referencias

Django Software Foundation. (2025). Django documentation (versión 5.2). https://docs.djangoproject.com/en/5.2/

Django Software Foundation. (2025). The Django admin site. https://docs.djangoproject.com/en/5.2/ref/contrib/admin/

Django Software Foundation. (2025). Customizing authentication in Django. https://docs.djangoproject.com/en/5.2/topics/auth/customizing/

Django Software Foundation. (2025). Migrations. https://docs.djangoproject.com/en/5.2/topics/migrations/

Encode OSS. (2025). Django REST framework: Authentication. https://www.django-rest-framework.org/api-guide/authentication/

Oracle Corporation. (2025). MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/

Oracle Corporation. (2026). MySQL 9.7 reference manual. https://dev.mysql.com/doc/refman/9.7/en/

Bootstrap Team. (2025). Bootstrap 5.3 documentation. https://getbootstrap.com/docs/5.3/

Docker Inc. (2025). Docker Compose overview. https://docs.docker.com/compose/

Congreso de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981